

看看使用了 `mb()` 和 `rmb()` 的一个例子，其中 `a` 的初始值是 1，`b` 的初始值是 2。

线程 1	线程 2
<code>a = 3;</code>	—
<code>mb();</code>	—
<code>b = 4;</code>	<code>c = b;</code>
—	<code>rmb();</code>
—	<code>d = a;</code>

如果不使用内存屏障，在某些处理器上，`c` 可能接收了 `b` 的新值，而 `d` 接收了 `a` 原来的值。比如 `c` 可能等于 4（正是我们希望的），然而 `d` 可能等于 1（不是我们希望的）。使用 `mb()` 能确保 `a` 和 `b` 按照预定的顺序写入，而 `rmb()` 确保 `c` 和 `d` 按照预定的顺序读取。

这种重排序的发生是因为现代处理器为了优化其传送管道（pipeline），打乱了分派和提交指令的顺序。如果上例中读入 `a`、`b` 时的顺序被打乱的话，又会发生什么情况呢？`rmb()` 或 `wmb()` 函数相当于指令，它们告诉处理器在继续执行前提交所有尚未处理的载入或存储指令。

看一个类似的例子，但是其中一个线程用 `read_barrier_depends()` 代替了 `rmb()`。例子中 `a` 的初始值是 1，`b` 是 2，`p` 是 `&b`。

线程 1	线程 2
<code>a = 3;</code>	—
<code>mb();</code>	—
<code>p = &a;</code>	<code>pp = p;</code>
—	<code>read_barrier_depends();</code>
—	<code>b = *pp;</code>

再一次声明，如果没有内存屏障，有可能在 `pp` 被设置成 `p` 前，`b` 就被设置为 `pp` 了。由于载入 `*pp` 依靠载入 `p`，所以 `read_barrier_depends()` 提供了一个有效的屏障。虽然使用 `rmb()` 同样有效，但是因为读是数据相关的，所以我们使用 `read_barrier_depends()` 可能更快。注意，不管在何种情况下，左边的线程都需要 `mb()` 操作来确保预定的载入或存储顺序。

宏 `smp_rmb()`、`smp_wmb()`、`smp_mb()` 和 `smp_read_barrier_depends()` 提供了一个有用的优化。在 SMP 内核中它们被定义成常用的内存屏障，而在单处理机内核中，它们被定义成编译器的屏障。对于 SMP 系统，在有顺序限定要求时，可以使用 SMP 的变种。

`barrier()` 方法可以防止编译器跨屏障对载入或存储操作进行优化。编译器不会重新组织存储或载入操作，而防止改变 C 代码的效果和现有数据的依赖关系。但是，它不知道在当前上下文之外会发生什么事。例如，编译器不可能知道有中断发生，这个中断有可能在读取正在被写入的数据。这时就要求存储操作发生在读取操作前。前面讨论的内存屏障可以完成编译器屏障的功能，但是编译器屏障要比内存屏障轻量（它实际上是轻快的）得多。实际上，编译器屏障几乎是空闲的，因为它只防止编译器可能重排指令。

表 10-12 给出了内核中所有体系结构提供的完整的内存和编译器屏障方法。

表 10-12 内存和编译器屏障方法

屏 障	描 述
rmb()	阻止跨越屏障的载入动作发生重排序
read_barrier_depends()	阻止跨越屏障的具有数据依赖关系的载入动作重排序
wmb()	阻止跨越屏障的存储动作发生重排序
mb()	阻止跨越屏障的载入和存储动作重新排序
smp_rmb()	在 SMP 上提供 rmb() 功能, 在 UP 上提供 barrier() 功能
smp_read_barrier_depends()	在 SMP 上提供 read_barrier_depends() 功能, 在 UP 上提供 barrier() 功能
smp_wmb()	在 SMP 上提供 wmb() 功能, 在 UP 上提供 barrier() 功能
smp_mb()	在 SMP 上提供 mb() 功能, 在 UP 上提供 barrier() 功能
barrier()	阻止编译器跨屏障对载入或存储操作进行优化

注意, 对于不同体系结构, 屏障的实际效果差别很大。例如, 如果一个体系结构不执行打乱存储 (如 Intel x86 芯片就不会), 那么 wmb() 就什么也不做。但应该为最坏的情况 (即排序能力最弱的处理器) 使用恰当的内存屏蔽, 这样代码才能在编译时执行针对体系结构的优化。

10.12 小结

本章应用了第 9 章的概念和原理, 这使得你能理解 Linux 内核用于同步和并发的具体方法。我们一开始先讲述了最简单的确保同步的方法——原子操作, 然后考察了自旋锁, 这是内核中最普通的锁, 它提供了轻量级单独持有者的锁, 即争用时忙等。我们接着还讨论了信号量 (这是一种睡眠锁) 以及更通用的衍生锁——mutex。至于专用的加锁原语像完成变量、seq 锁, 只是稍稍提及。我们取笑 BLK, 考察了禁止抢占, 并理解了屏障, 它曾难以驾驭。

以第 9 章和第 10 章的同步方法为基础, 就可以编写避免竞争条件、确保正确同步, 而且能在多处理器上安全运行的内核代码了。



第 11 章

定时器和时间管理

时间管理在内核中占有非常重要的地位。相对于事件驱动[⊖]而言，内核中有大量的函数都是基于时间驱动的。其中有些函数是周期执行的，像对调度程序中的运行队列进行平衡调整或对屏幕进行刷新这样的函数，都需要定期执行，比如说，每秒执行 100 次；而另外一些函数，比如需要推后执行的磁盘 I/O 操作等，则需要等待一个相对时间后才运行——比如说，内核会在 500ms 后再执行某个任务。除了上述两种函数需要内核提供时间外，内核还必须管理系统的运行时间以及当前日期和时间。

请注意相对时间和绝对时间之间的差别。如果某个事件在 5s 后被调度执行，那么系统所需要的不是绝对时间，而是相对时间（比如，相对现在起 5s 后）；相反，如果要求管理当前日期和当前时间，则内核不但要计算流逝的时间而且还要计算绝对时间。所以这两种时间概念对内核时间管理来说都至关重要。

另外，还请注意周期性产生的事件与内核调度程序推迟到某个确定点执行的事件之间的差别。周期性产生的事件——比如每 10ms 一次——都是由系统定时器驱动的。系统定时器是一种可编程硬件芯片，它能以固定频率产生中断。该中断就是所谓的定时器中断，它所对应的中断处理程序负责更新系统时间，也负责执行需要周期性运行的任务。系统定时器和时钟中断处理程序是 Linux 系统内核管理机制中的中枢，本章将着重讨论它们。

本章关注的另外一个焦点是动态定时器——一种用来推迟执行程序的工具。比如说，如果软驱马达在一定时间内都未活动，那么软盘驱动程序会使用动态定时器关闭软驱马达。内核可以动态创建或撤销动态定时器。本章将介绍动态定时器在内核中的实现，同时给出在内核代码中可供使用的定时器接口。

11.1 内核中的时间概念

时间概念对计算机来说有些模糊，事实上内核必须在硬件的帮助下才能计算和管理时间。硬件为内核提供了一个系统定时器用以计算流逝的时间，该时钟在内核中可看成是一个电子时间资源，比如数字时钟或处理器频率等。系统定时器以某种频率自行触发（经常被称为击中（hitting）或射中（popping））时钟中断，该频率可以通过编程预定，称作节拍率（tick rate）。当时钟中断发生时，内核就通过一种特殊的中断处理程序对其进行处理。

因为预编的节拍率对内核来说是可知的，所以内核知道连续两次时钟中断的间隔时间。这

⊖ 更准确地讲，时间驱动事件也属于事件驱动的一种——时间的流逝本身就是一种事件。然而，由于时间驱动的频率非常高，且对内核而言至关重要，因此，本章中我们仅仅分析时间驱动事件。

个间隔时间就称为节拍 (tick)，它等于节拍率分之一 ($1/(\text{tick rate})$) 秒。正如你所看到的，内核就是靠这种已知的时钟中断间隔来计算墙上时间和系统运行时间的。墙上时间 (也就是实际时间) 对用户空间的应用程序来说是最重要的。内核通过控制时钟中断维护实际时间，另外内核也为用户空间提供了一组系统调用以获取实际日期和实际时间。系统运行时间 (自系统启动开始所经的时间) 对用户空间和内核都很有用，因为许多程序都必须清楚流逝的时间。通过两次 (现在和以后) 读取运行时间再计算它们的差，就可以得到相对的流逝的时间了。

- 时钟中断对于管理操作系统尤为重要，大量内核函数的生命周期都离不开流逝的时间的控制。下面给出一些利用时间中断周期执行的工作：
- 更新系统运行时间。
- 更新实际时间。
- 在 smp 系统上，均衡调度程序中各处理器上的运行队列。如果运行队列负载不均衡的话，尽量使它们均衡 (在第 4 章中讨论过)。
- 检查当前进程是否用尽了自己的时间片。如果用尽，就重新进行调度 (在第 4 章中讨论过)。
- 运行超时的动态定时器。
- 更新资源消耗和处理器时间的统计值。

这其中有些工作在每次的时钟中断处理程序中都要被处理——也就是说，这些工作随时钟的频率反复运行。另一些也是周期性地执行，但只需要每 n 次时钟中断运行一次，也就是说，这些函数在累计了一定数量的时钟节拍数时才被执行。在“定时器中断处理程序”这一小节中，我们将详细讨论时钟中断处理程序。

11.2 节拍率：HZ

系统定时器频率 (节拍率) 是通过静态预处理定义的，也就是 HZ (赫兹)，在系统启动时按照 HZ 值对硬件进行设置。体系结构不同，HZ 的值也不同，实际上，对于某些体系结构来说，甚至是机器不同，它的值都会不一样。

内核在 `<asm/param.h>` 文件中定义了这个值。节拍率有一个 HZ 频率，一个周期为 $1/\text{HZ}$ 秒。例如，x86 体系结构中，系统定时器频率默认值为 100。因此，x86 上时钟中断的频率就为 100HZ，也就是说在 i386 处理上的每秒钟时钟中断 100 次 (百分之一秒，即每 10ms 产生一次)。但其他体系结构的节拍率为 250 和 1000，分别对应 4ms 和 1ms。表 11-1 给出了各种体系结构与各自对应节拍率的完整列表。

编写内核代码时，不要认为 HZ 值是一个固定不变的值。这不是一个常见的错误，因为大多数体系结构的节拍率都是可调的。但是在过去，只有 Alpha 一种机型的节拍率不等于 100，所以很多本该使用 HZ 的地方，都错误地在代码中直接硬编码 (hard-code) 成 100 这个值。稍后，我们会给出内核代码中使用 HZ 的例子。

正如我们所看到的，时钟中断能处理许多内核任务，所以它对内核来说极为重要。事实上，内核中的全部时间概念都来源于周期运行的系统时钟。所以选择一个合适的频率，就如同在人际交往中建立和谐关系一样，必须取得各方面的折中。

表 11-1 时钟中断频率

体系结构	频率 /HZ
Alpha	1024
Arm	100
avr32	100
Blackfin	100
Cris	100
h8300	100
ia64	1 024
m32r	100
m68k	100
m68knommu	50, 100 或 1000
Microblaze	100
Mips	100
mn10300	100
parisc	100
powerpc	100
Score	100
s390	100
Sh	100
sparc	100
Um	100
x86	100

11.2.1 理想的 HZ 值

自 Linux 问世以来, i386 体系结构中时钟中断频率就设定为 100HZ, 但是在 2.5 开发版内核中, 中断频率被提高到 1000HZ。当然, 是否应该提高频率(如同其他绝大多数事情一样)是饱受争议的。由于内核中众多子系统都必须依赖时钟中断工作, 所以改变中断频率必然会对整个系统造成很大的冲击。但是, 任何事情总是有两面性的, 我们接下来就来分析系统定时器使用高频率与使用低频率各有哪些优劣。

提高节拍率意味着时钟中断产生得更加频繁, 所以中断处理程序也会更频繁地执行。如此一来会给整个系统带来如下好处:

- 更高的时钟中断解析度(resolution)可提高时间驱动事件的解析度。
- 提高了时间驱动事件的准确度(accuracy)。

提高节拍率等同于提高中断解析度。比如 HZ=100 的时钟的执行粒度为 10ms, 即系统中的周期事件最快为每 10ms 运行一次, 而不可能有更高的精度^①, 但是当 HZ=1000 时, 解析度就为

^① 这里所说的是计算机意义上的精度, 而不是科学意义上的精度。科学意义上的精度是统计反复性的量度, 在计算机领域内, 精度是表示一个值的有效位的个数。

1ms——精细了 10 倍。虽然内核可以提供频度为 1ms 的时钟，但是并没有证据显示对系统中所有程序而言，频率为 1000Hz 的时钟率相比频率为 100Hz 的时钟都更合适。

另外，提高解析度的同时也提高了准确度。假定内核在某个随机时刻触发定时器，而它可能在任何时间超时，但由于只有在时钟中断到来时才可能执行它，所以平均误差大约为半个时钟中断周期。比如说，如果时钟周期为 $HZ=100$ ，那么事件平均在设定时刻的 $\pm 5ms$ 内发生，所以平均误差为 5ms。如果 $HZ=1000$ ，那么平均误差可降低到 0.5ms——准确度提高了 10 倍。

11.2.2 高 HZ 的优势

更高的时钟中断频度和更高的准确度又会带来如下优点：

- 内核定时器能够以更高的频度和更高的准确度（它带来了大量的好处，下一条便是其中之一）运行。
- 依赖定时值执行的系统调用，比如 `poll()` 和 `select()`，能够以更高的精度运行。
- 对诸如资源消耗和系统运行时间等的测量会有更精细的解析度。
- 提高进程抢占的准确度。

对 `poll()` 和 `select()` 超时精度的提高会给系统性能带来极大的好处。提高精度可以大幅度提高系统性能。频繁使用上述两种系统调用的应用程序，往往在等待时钟中断上浪费大量的时间，而事实上，定时值可能早就超时了。回忆一下，平均误差（也就是，可能浪费的时间）可是时钟中断周期的一半。

更高的准确率也使进程抢占更准确，同时还会加快调度响应时间。第 4 章中提到过，时钟中断处理程序负责减少当前进程的时间片计数。当时间片计数跌到 0 时，而又设置了 `need_resched` 标志的话，内核便立刻重新运行调度程序。假定有一个正在运行的进程，它的时间片只剩下 2ms 了，此时调度程序又要求抢占该进程，然后去运行另一个新进程；然而，该抢占行为不会在下一个时钟中断到来前发生，也就是说，在这 2ms 内不可能进行抢占。实际上，对于频率为 100Hz 的时钟来说，最坏要在 10ms 后，当下一个时钟中断到来时才能进行抢占，所以新进程也就可能要比要求的晚 10ms 才能执行。当然，进程之间也是平等的，因为所有的进程都是一视同仁的待遇，调度起来都不是很准确——但关键不在于此。问题在于由于耽误了抢占，所以对于类似于填充音频缓冲区这样有严格时间要求的任务来说，结果是无法接受的。如果将节拍率提高到 1000Hz，在最坏情况下，也能将调度延误时间降低到 1ms，而在平均情况下，只能降到 0.5ms 左右。

11.2.3 高 HZ 的劣势

现在该谈谈另一面了，提高节拍率会产生副作用。事实上，把节拍率提高到 1000Hz（甚至更高）会带来一个大问题：节拍率越高，意味着时钟中断频率越高，也就意味着系统负担越重。因为处理器必须花时间来执行时钟中断处理程序，所以节拍率越高，中断处理程序占用的处理器的时间越多。这样不但减少了处理器处理其他工作的时间，而且还会更频繁地打乱处理器高速缓存并增加耗电。负载造成的影响值得进一步探讨。将时钟频率从 100Hz 提高到 1000Hz 必然会使时钟中断的负载增加 10 倍。可是增加前的系统负载又是多少呢？最后的结论是：至少在现代计算机系统上，时钟频率为 1000Hz 不会导致难以接受的负担，并且不会对系统性能造成较大的

影响。尽管如此，在 2.6 版内核中还是允许在编译内核时选定不同的 HZ 值^①。

无节拍的 OS?

也许你疑惑操作系统是否一定要有固定时钟。尽管 40 年来，几乎所有的通用操作系统都使用与本章所描述的系统类似的时钟中断，但 Linux 内核支持“无节拍操作”这样的选项。当编译内核时设置了 CONFIG_HZ 配置选项，系统就根据这个选项动态调度时钟中断。并不是每隔固定的时间间隔（比如 1ms）触发时钟中断，而是按需动态调度和重新设置。如果下一个时钟频率设置为 3ms，就每 3ms 触发一次时钟中断。之后，如果 50ms 内都无事可做，内核以 50ms 重新调度时钟中断。

减少开销总是受欢迎的，但是实质性受益还是省电，特别是在系统空闲时。在基于节拍的标准系统中，即使在系统空闲期间，内核也需要为时钟中断提供服务。对于无节拍的系统而言，空闲档期不会被不必要的时钟中断所打断，于是减少了系统的能耗。且不论空闲期是 200ms 还是 200 秒，随着时间的推移，所省的电是实实在在的。

11.3 jiffies

全局变量 jiffies 用来记录自系统启动以来产生的节拍的总数。启动时，内核将该变量初始化为 0，此后，每次时钟中断处理程序就会增加该变量的值。因为一秒内时钟中断的次数等于 HZ，所以 jiffies 一秒内增加的值也就为 HZ。系统运行时间以秒为单位计算，就等于 jiffies/HZ。实际出现的情况可能稍微复杂些：内核给 jiffies 赋一个特殊的初值，引起这个变量不断地溢出，由此捕捉 bug。当找到实际的 jiffies 值后，就首先把这个“偏差”减去。

Jiffy 的语源

术语 jiffy 起源是未知的。据说这个短语起源于 18 世纪的英国。最初，jiffy 所指含义不明确，但简单地表示时间周期。

在科学应用中，jiffy 表示各种时间间隔，通常指 10ms。在物理中，jiffy 有时表示光传播某一特定距离（大抵 1 英尺，或者 1 厘米，或者跨越 1 个核子）所花的时间。

在计算机工程中，jiffy 常常是两次连续的时钟周期之间的时间。在电机工程中，jiffy 是完成一次 AC（交流电）周期的时间。在美国，这是 1/60 秒。

在操作系统中，尤其是 Unix 中，jiffy 是两次连续的时钟节拍之间的时间。历史上，这是 10ms。但是，我们在本章已经看到，jiffy 在 Linux 中已经有所变化。

jiffies 定义于文件 <linux/jiffies.h> 中：

```
extern unsigned long volatile jiffies;
```

在 13.4 节我们会看到它的实际定义，它看起来有点特殊。现在我们先来看一些用到 jiffies 的内核代码。下面表达式将以秒为单位的时间转化为 jiffies：

^① 不过，因为体系结构和 NTP 相关问题，HZ 的值并不是随便确定的，在 x86 上，100、500 和 1000 都是有效的值。

(seconds * HZ)

相反，下面表达式将 jiffies 转换为以秒为单位的时间：

(jiffies/HZ)

比较而言，内核中将秒转换为 jiffies 用得更多一些，比如代码经常需要设置一些将来的时间：

```
unsigned long time_stamp = jiffies;          /* 现在 */
unsigned long next_tick = jiffies+1;         /* 从现在开始 1 个节拍 */
unsigned long later = jiffies+5*HZ;          /* 从现在开始 5 秒 */
unsigned long fraction = jiffies + HZ / 10;  /* 从现在开始 1/10 秒 */
```

把时钟转化为秒经常会用在内核和用户空间进行交互的时候，而内核本身很少用到绝对时间。注意，jiffies 类型为无符号长整型 (unsigned long)，用其他任何类型存放它都不正确。

11.3.1 jiffies 的内部表示

jiffies 变量总是无符号长整数 (unsigned long)，因此，在 32 位体系结构上是 32 位，在 64 位体系结构上是 64 位。32 位的 jiffies 变量，在时钟频率为 100HZ 的情况下，497 天后会溢出。如果频率为 1000HZ，49.7 天后就会溢出。而如果使用 64 位的 jiffies 变量，任何人都别指望会看到它溢出。

由于性能与历史的原因，主要还考虑到与现有内核代码的兼容性，内核开发者希望 jiffie 依然为 unsigned long。有一些巧妙的思想和少数神奇的链接程序扭转了这一局面。

前面已经看到，jiffies 定义为 unsigned long：

```
extern unsigned long volatile jiffies;
```

第二个变量也定义在 <linux/jiffies.h> 中：

```
extern u64 jiffies_64;
```

ld(1) 脚本用于连接主内核映像（在 x86 上位于 arch/x86/kernel/vmlinux.lds.S），然后用 jiffies_64 变量的初值覆盖 jiffies 变量：

```
jiffies = jiffies_64;
```

因此，jiffies 取整个 64 位 jiffies_64 变量的低 32 位。代码可以完全像以前一样继续访问 jiffies。因为大多数代码只不过使用 jiffies 存放流失的时间，因此，也就只关心低 32 位。不过，时间管理代码使用整个 64 位，以此来避免整个 64 位的溢出。图 11-1 呈现了 jiffies 和 jiffies_64 的划分。

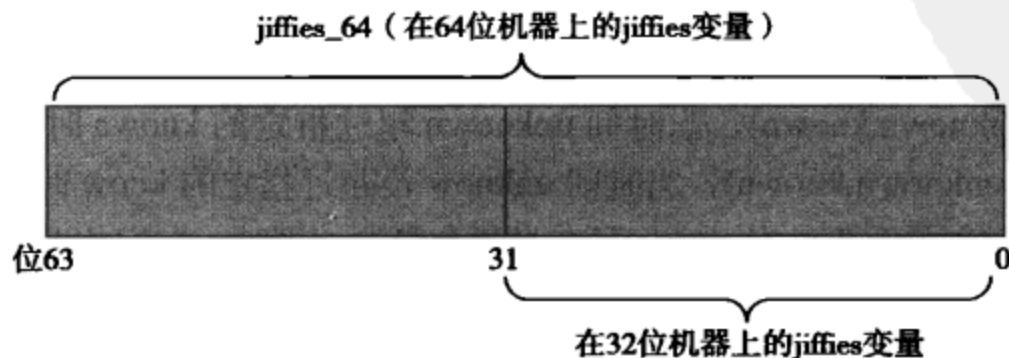


图 11-1 jiffies 和 jiffies_64 的划分

访问 `jiffies` 的代码仅会读取 `jiffies_64` 的低 32 位。通过 `get_jiffies_64()` 函数，就可以读取整个 64 位数值^①。但是这种需求很少，多数代码仍然只要能通过 `jiffies` 变量读取低 32 位就够了。

在 64 位体系结构上，`jiffies_64` 和 `jiffies` 指的是同一个变量，代码既可以直接读取 `jiffies` 也可以调用 `get_jiffies_64()` 函数，它们的作用相同。

11.3.2 `jiffies` 的回绕

和任何 C 整型一样，当 `jiffies` 变量的值超过它的最大存放范围后就会发生溢出。对于 32 位无符号长整型，最大取值为 $2^{32}-1$ 。所以在溢出前，定时器节拍计数最大为 4294967295。如果节拍计数达到了最大值后还要继续增加的话，它的值会回绕 (wrap around) 到 0。

请看下面一个回绕的例子：

```
unsigned long timeout = jiffies + HZ/2;    /* 0.5 秒后超时 */
/* 执行一些任务 ... */

/* 然后查看是否花的时间过长 */
if(timeout > jiffies){
    /* 没有超时，很好 ... */
}else {
    /* 超时了，发生错误 ... */
}
```

上面这一小段代码是希望设置一个准确的超时时间——本例中从现在开始计时，时间为半秒。然后再去处理一些工作，比如探测硬件然后等待它的响应。如果处理这些工作的时间超过了设定的超时时间，代码就要做相应的出错处理。

这里有很多种发生溢出的可能，我们只分析其中之一：考虑如果在设置完 `timeout` 变量后，`jiffies` 重新回绕为 0 将会发生什么？此时，第一个判断会返回假，因为尽管实际上用去的时间可能比 `timeout` 值要大，但是由于溢出后回绕为 0，所以 `jiffies` 这时肯定会小于 `timeout` 的值。`jiffies` 本该是个非常大的数值——大于 `timeout`，但是因为超过了它的最大值，所以反而变成了一个很小的值——也许仅仅只有几个节拍计数。由于发生了回绕，所以 `if` 判断语句的结果刚好相反。

幸好，内核提供了四个宏来帮助比较节拍计数，它们能正确地处理节拍计数回绕情况。这些宏定义在文件 `<linux/jiffies.h>` 中，这里列出的宏是简化版：

```
#define time_after(unknown,known) ((long)(known) - (long)(unknown)<0)
#define time_before(unknown,known) ((long)(unknown) - (long)(known)<0)
#define time_after_eq(unknown,known) ((long)(unknown) - (long)(known)>=0)
#define time_before_eq(unknown,known) ((long)(known) - (long)(unknown)>=0)
```

其中 `unknown` 参数通常是 `jiffies`，`known` 参数是需要对比的值。

宏 `time_after(unknown,known)`，当时间 `unknown` 超过指定的 `known` 时，返回真，否则返回假；宏 `time_before(unknown,known)`，当时间 `unknown` 没超过指定的 `known` 时，返回真，否则返回假。后面两个宏作用和前面两个宏一样，只有当两个参数相等时，它们才返回真。

^① 因为 32 位体系结构不能原子地一次访问 64 位变量中的两个 32 位数值。在读取 `jiffies` 时，特殊的函数利用 `xtime_lock` 锁对 `jiffies` 变量进行锁定。

所以前面的例子可以改造成时钟一回绕一安全 (timer-wraparound-safe) 的版本, 形式如下:

```
unsigned long timeout = jiffies + HZ/2 ;      /* 0.5 秒后超时 */

/*...*/
if(time_before(jiffies,timeout)){
    /* 没有超时, 很好 ...*/
}else {
    /* 超时了, 发生错误 ...*/
}
```

如果你对这些宏能避免因为回绕而产生的错误感到好奇的话, 你可以试一试对这两个参数取不同的值。然后, 设定一个参数回绕到 0 值, 看看会发生什么。

11.3.3 用户空间和 HZ

在 2.6 版以前的内核中, 如果改变内核中 HZ 的值, 会给用户空间中某些程序造成异常结果。这是因为内核是以节拍数 / 秒的形式给用户空间导出这个值的, 在这个接口稳定了很长一段时间后, 应用程序便逐渐依赖于这个特定的 HZ 值了。所以如果在内核中更改了 HZ 的定义值, 就打破了用户空间的常量关系——用户空间并不知道新的 HZ 值。所以用户空间可能认为系统运行时间已经是 20 个小时了, 但实际上系统仅仅启动了两个小时。

要想避免上面的错误, 内核必须更改所有导出的 jiffies 值。因而内核定义了 USER_HZ 来代表用户空间看到的 HZ 值。在 x86 体系结构上, 由于 HZ 值原来一直是 100, 所以 USER_HZ 值就定义为 100。内核可以使用函数 jiffies_to_clock_t() (定义于 kernel/time.c 中) 将一个由 HZ 表示的节拍计数转换成一个由 USER_HZ 表示的节拍计数。所采用的表达式取决于 USER_HZ 和 HZ 是否互为整数倍, 而且 USER_HZ 是否小于等于 HZ。如果这两个条件都满足, 对大多数系统来说通常也能够满足, 则表达式相当简单:

```
return x / (HZ / USER_HZ);
```

如果不是整数倍关系, 那么该宏就得用到更为复杂的算法了。

最后还要说明, 内核使用函数 jiffies_64_to_clock_t() 将 64 位的 jiffies 值的单位从 HZ 转换为 USER_HZ。

在需要把以节拍数 / 秒为单位的值导出到用户空间时, 需要使用上面这几个函数。比如:

```
unsigned long start ;
unsigned long total_time;

start = jiffies;
/* 执行一些任务 ...*/
total_time = jiffies - start;
printf("That took %lu ticks\n", jiffies_to_clock_t(total_time));
```

用户空间期望 HZ=USER_HZ, 但是如果它们不相等, 则由宏完成转换, 这样的结果自然是皆大欢喜。说实话, 上面的例子看起来是挺简单的, 如果以秒为单位而不是以节拍为单位, 输出信息会执行得好一些。比如像下面这样:

```
printk("That took %lu seconds \n",total_time/HZ);
```

11.4 硬时钟和定时器

体系结构提供了两种设备进行计时——一种是我们前面讨论过的系统定时器；另一种是实时时钟。虽然在不同机器上这两种时钟的实现并不相同，但是它们有着相同的作用和设计思路。

11.4.1 实时时钟

实时时钟（RTC）是用来持久存放系统时间的设备，即便系统关闭后，它也可以靠主板上的微型电池提供的电力保持系统的计时。在 PC 体系结构中，RTC 和 CMOS 集成在一起，而且 RTC 的运行和 BIOS 的保存设置都是通过同一个电池供电的。

当系统启动时，内核通过读取 RTC 来初始化墙上时间，该时间存放在 xtime 变量中。虽然内核通常不会在系统启动后再读取 xtime 变量，但是有些体系结构（比如 x86）会周期性地将当前时间值存回 RTC 中。尽管如此，实时时钟最主要的作用仍是在启动时初始化 xtime 变量。

11.4.2 系统定时器

系统定时器是内核定时机制中最为重要的角色。尽管不同体系结构中的定时器实现不尽相同，但是系统定时器的根本思想并没有区别——提供一种周期性触发中断机制。有些体系结构是通过电子晶振进行分频来实现系统定时器，还有些体系结构则提供了一个衰减测量器（decrementer）——衰减测量器设置一个初始值，该值以固定频率递减，当减到零时，触发一个中断。无论哪种情况，其效果都一样。

在 x86 体系结构中，主要采用可编程中断时钟（PIT）。PIT 在 PC 机器中普遍存在，而且从 DOS 时代，就开始以它作为时钟中断源了。内核在启动时对 PIT 进行编程初始化，使其能够以 HZ/秒的频率产生时钟中断（中断 0）。虽然 PIT 设备很简单，功能也有限，但它却足以满足我们的需要。x86 体系结构中的其他的时钟资源还包括本地 APIC 时钟和时间戳计数（TSC）等。

11.5 时钟中断处理程序

现在我们已经理解了 HZ、jiffies 等概念以及系统定时器的功能。下面将分析时钟中断处理程序是如何实现的。时钟中断处理程序可以划分为两个部分：体系结构相关部分和体系结构无关部分。

与体系结构相关的例程作为系统定时器的中断处理程序而注册到内核中，以便在产生时钟中断时，它能够相应地运行。虽然处理程序的具体工作依赖于特定的体系结构，但是绝大多数处理程序最低限度也都要执行如下工作：

- 获得 xtime_lock 锁，以便对访问 jiffies_64 和墙上时间 xtime 进行保护。
- 需要时应答或重新设置系统时钟。
- 周期性地使用墙上时间更新实时时钟。
- 调用体系结构无关的时钟例程：tick_periodic()。

中断服务程序主要通过调用与体系结构无关的例程，tick_periodic() 执行下面更多的工作：

- 给 `jiffies_64` 变量增加 1（这个操作即使是在 32 位体系结构上也是安全的，因为前面已经获得了 `xtime_lock` 锁）。
- 更新资源消耗的统计值，比如当前进程所消耗的系统时间和用户时间。
- 执行已经到期的动态定时器（11.6 节将讨论）。
- 执行第 4 章曾讨论的 `scheduler_tick()` 函数。
- 更新墙上时间，该时间存放在 `xtime` 变量中。
- 计算平均负载值。

因为上述工作分别都由单独的函数负责完成，所以 `tick_periodic()` 例程的代码看起来非常简单。

```
static void tick_periodic(int cpu)
{
    if (tick_do_timer_cpu == cpu) {
        write_seqlock(&xtime_lock);

        /* 记录下一个节拍事件 */
        tick_next_period = ktime_add(tick_next_period, tick_period);

        do_timer(1);
        write_sequnlock(&xtime_lock);
    }

    update_process_times(user_mode(get_irq_regs()));
    profile_tick(CPU_PROFILING);
}
```

很多重要的操作都在 `do_timer()` 和 `update_process_times()` 函数中进行。前者承担着对 `jiffies_64` 的实际增加操作：

```
void do_timer(unsigned long ticks)
{
    jiffies_64 += ticks;
    update_wall_time();
    calc_global_load();
}
```

函数 `update_wall_time()`，顾名思义，根据所流逝的时间更新墙上的时钟，而 `calc_global_load()` 更新系统的平均负载统计值。当 `do_timer()` 最终返回时，调用 `update_process_times()` 更新所耗费的各种节拍数。注意，通过 `user_tick` 区别是花费在用户空间还是内核空间。

```
void update_process_times(int user_tick)
{
    struct task_struct *p = current;
    int cpu = smp_processor_id();
    /* 注意：也必须对这个时钟 irq 的上下文说明一下原因 */
    account_process_tick(p, user_tick);
    run_local_timers();
    rcu_check_callbacks(cpu, user_tick);
    printk_tick();
    scheduler_tick();
    run_posix_cpu_timers(p);
}
```

回想一下 `tick_periodic()`, `user_tick` 的值是通过查看系统寄存器来设置的:

```
update_process_times(user_mode(get_irq_regs()));
```

`account_process_tick()` 函数对进程的时间进行实质性更新:

```
void account_process_tick(struct task_struct *p, int user_tick)
{
    cputime_t one_jiffy_scaled = cputime_to_scaled(cputime_one_jiffy);
    struct rq *rq = this_rq();

    if (user_tick)
        account_user_time(p, cputime_one_jiffy, one_jiffy_scaled);
    else if ((p != rq->idle) || (irq_count() != HARDIRQ_OFFSET))
        account_system_time(p, HARDIRQ_OFFSET, cputime_one_jiffy,
                             one_jiffy_scaled);
    else
        account_idle_time(cputime_one_jiffy);
}
```

也许你已经发现了, 这样做意味着内核对进程进行时间计数时, 是根据中断发生时处理器所处的模式进行分类统计的, 它把上一个节拍全部算给了进程。但是事实上进程在上一个节拍期间可能多次进入和退出内核模式, 而且在上一个节拍期间, 该进程也不一定是唯一一个运行进程。很不幸, 这种粒度的进程统计方式是传统的 Unix 所具有的, 现在还没有更加精密的统计算法的支持, 内核现在只能做到这个程度。这也是内核应该采用更高频率的另一个原因。

接下来的 `run_lock_timers()` 函数标记了一个软中断 (请参考第 8 章) 去处理所有到期的定时器, 在 11.6 节中将具体讨论定时器。

最后, `scheduler_tick()` 函数负责减少当前运行进程的时间片计数值并且在需要时设置 `need_resched` 标志。在 SMP 机器中, 该函数还要负责平衡每个处理器上的运行队列, 这点在第 4 章曾讨论过。

`tick_periodic()` 函数执行完毕后返回与体系结构相关的中断处理程序, 继续执行后面的工作, 释放 `xtime_lock` 锁, 然后退出。

以上全部工作每 1/HZ 秒都要发生一次, 也就是说在 x86 机器上时钟中断处理程序每秒执行 100 次或者 1000 次。

11.6 实际时间

当前实际时间 (墙上时间) 定义在文件 `kernel/time/timekeeping.c` 中:

```
struct timespec xtime;
```

`timespec` 数据结构定义在文件 `<linux/time.h>` 中, 形式如下:

```
struct timespec{
    _kernel_time_t tv_sec;        /* 秒 */
    long tv_nsec;                /* ns */
};
```


`xtime.tv_sec` 以秒为单位，存放着自 1970 年 1 月 1 日 (UTC) 以来经过的时间，1970 年 1 月 1 日被称为纪元，多数 Unix 系统的墙上时间都是基于该纪元而言的。`xtime.v_nsec` 记录自上一秒开始经过的 ns 数。

读写 `xtime` 变量需要使用 `xtime_lock` 锁，该锁不是普通自旋锁而是一个 seqlock 锁，在第 10 章中曾讨论过 seqlock 锁。

更新 `xtime` 首先要申请一个 seqlock 锁：

```
write_seqlock(&xtime_lock);
/* 更新 xtime... */
write_sequnlock(&xtime_lock);
```

读取 `xtime` 时也要使用 `read_seqbegin()` 和 `read_seqretry()` 函数：

```
unsigned long seq;

do {
    unsigned long lost;
    seq = read_seqbegin(&xtime_lock);

    usec = timer->get_offset();
    lost = jiffies - wall_jiffies;
    if (lost)
        usec += lost * (1000000 / HZ);
    sec = xtime.tv_sec;
    usec += (xtime.tv_nsec / 1000);
} while (read_seqretry(&xtime_lock, seq));
```

该循环不断重复，直到读者确认读取数据时没有写操作介入。如果发现循环期间有时钟中断处理程序更新 `xtime`，那么 `read_seqretry()` 函数就返回无效序列号，继续循环等待。

从用户空间取得墙上时间的主要接口是 `gettimeofday()`，在内核中对应系统调用为 `sys_gettimeofday()`，定义于 `kernel/time.c`：

```
asmlinkage long sys_gettimeofday(struct timeval *tv, struct timezone *tz)
{
    if (likely(tv)) {
        struct timeval ktv;
        do_gettimeofday(&ktv);
        if (copy_to_user(tv, &ktv, sizeof(ktv)))
            return -EFAULT;
    }
    if (unlikely(tz)) {
        if (copy_to_user(tz, &sys_tz, sizeof(sys_tz)))
            return -EFAULT;
    }
    return 0;
}
```

如果用户提供的 tv 参数非空，那么与体系结构相关的 do_gettimeofday() 函数将被调用。该函数执行的就是上面提到的循环读取 xtime 的操作。如果 tz 参数为空，该函数将把系统时区（存放在 sys_tz 中）返回用户。如果在给用户空间拷贝墙上时间或时区时发生错误，该函数返回 -EFAULT；如果成功，则返回 0。

虽然内核也实现了 time() [⊖] 系统调用，但是 gettimeofday() 几乎完全取代了它。另外 C 库函数也提供了一些墙上时间相关的库调用，比如 ftime() 和 ctime()。

另外，系统调用 settimeofday() 来设置当前时间，它需要具有 CAP_SYS_TIME 权能。

除了更新 xtime 时间以外，内核不会像用户空间程序那样频繁使用 xtime。但也有需要注意的特殊情况，那就是在文件系统的实现代码中存放访问时间戳（创建、存取、修改等）时需要使用 xtime。

11.7 定时器

定时器（有时也称为动态定时器或内核定时器）是管理内核流逝的时间的基础。内核经常需要推迟执行某些代码，比如以前章节提到的下半部机制就是为了将工作放到以后执行。但不幸的是，之后这个概念很含糊，下半部的本意并非是放到以后的某个时间去执行任务，而仅仅是不在当前时间执行就可以了。我们所需要的是一种工具，能够使工作在指定时间点上执行——不长不短，正好在希望的时间点上。内核定时器正是解决这个问题的理想工具。

定时器的使用很简单。你只需要执行一些初始化工作，设置一个超时时间，指定超时发生后执行的函数，然后激活定时器就可以了。指定的函数将在定时器到期时自动执行。注意定时器并不周期运行，它在超时后就自行撤销，这也正是这种定时器被称为动态定时器 [⊖] 的一个原因；动态定时器不断地创建和撤销，而且它的运行次数也不受限制。定时器在内核中应用得非常普遍。

11.7.1 使用定时器

定时器由结构 timer_list 表示，定义在文件 <linux/timer.h> 中。

```
struct timer_list {
    struct list_head entry;           /* 定时器链表的入口 */
    unsigned long expires;           /* 以 jiffies 为单位的定时值 */
    void (*function)(unsigned long); /* 定时器处理函数 */
    unsigned long data;              /* 传给处理函数的长整型参数 */
    struct tvec_t_base_s *base;      /* 定时器内部值，用户不要使用 */
};
```

幸运的是，使用定时器并不需要深入了解该数据结构。事实上，过深地陷入该结构，反而会使你的代码不能保证对可能发生的变化提供支持。内核提供了一组与定时器相关的接口用来简化管理定时器的操作。所有这些接口都声明在文件 <linux/timer.h> 中，大多数接口在文件 kernel/timer.c 中获得实现。

⊖ 但在某些体系结构中，并没有实现 sys_time()，而是用 C 库中的 gettimeofday() 函数模拟它。

⊖ 另一个原因是（2.3 版本前）内核也存在静态定时器。这种定时器在编译时创建，而不是实时创建。由于静态定时器存在缺陷，已经被淘汰了。

创建定时器时需要先定义它：

```
struct timer_list my_timer;
```

接着需要通过一个辅助函数来初始化定时器数据结构的内部值，初始化必须在使用其他定时器管理函数对定时器进行操作前完成。

```
init_timer(&my_timer);
```

现在你可以填充结构中需要的值了：

```
my_timer.expires = jiffies + delay;    /* 定时器超时的节拍数 */
my_timer.data = 0;                    /* 给定时器处理函数传入 0 值 */
my_timer.function = my_function;      /* 定时器超时时调用的函数 */
```

`my_timer.expires` 表示超时时间，它是以节拍为单位的绝对计数值。如果当前 `jiffies` 计数等于或大于 `my_timer.expires`，那么 `my_timer.function` 指向的处理函数就会开始执行，另外该函数还要使用长整型参数 `my_timer.data`。所以正如我们从 `timer_list` 结构看到的形式，处理函数必须符合下面的函数原型：

```
void my_timer_function(unsigned long data);
```

`data` 参数使你可以利用同一个处理函数注册多个定时器，只需通过该参数就能区别对待它们。如果你不需要这个参数，就可以简单地传递 0（或任何其他值）给处理函数。

最后，你必须激活定时器：

```
add_timer(&my_timer);
```

大功告成，定时器可以工作了！但请注意定时值的重要性。当前节拍计数等于或大于指定的超时时，内核就开始执行定时器处理函数。虽然内核可以保证不会在超时时间到期前运行定时器处理函数，但是有可能延误定时器的执行。一般来说，定时器都在超时后马上就会执行，但是也有可能推迟到下一次时钟节拍时才能运行，所以不能用定时器来实现任何硬实时任务。

有时可能需要更改已经激活的定时器超时时间，所以内核通过函数 `mod_timer()` 来实现该功能，该函数可以改变指定的定时器超时时间：

```
mod_timer(&my_timer, jiffies+new_delay);    /* 新的定时值 */
```

`mod_timer()` 函数也可操作那些已经初始化，但还没有被激活的定时器，如果定时器未被激活，`mod_timer()` 会激活它。如果调用时定时器未被激活，该函数返回 0；否则返回 1。但不论哪种情况，一旦从 `mod_timer()` 函数返回，定时器都将被激活而且设置了新的定时值。

如果需要在定时器超时前停止定时器，可以使用 `del_timer()` 函数：

```
del_timer(&my_timer);
```

被激活或未被激活的定时器都可以使用该函数，如果定时器还未被激活，该函数返回 0；否则返回 1。注意，不需要为已经超时的定时器调用该函数，因为它们会自动删除。

当删除定时器时，必须注意一个潜在的竞争条件。当 `del_timer()` 返回后，可以保证的只是：定时器不会再被激活（也就是，将来不会执行），但是在多处理器机器上定时器中断可能已经在

其他处理器上运行了，所以删除定时器时需要等待可能在其他处理器上运行的定时器处理程序都退出，这时就要使用 `del_timer_sync()` 函数执行删除工作：

```
del_timer_sync(&my_timer);
```

和 `del_timer()` 函数不同，`del_timer_sync()` 函数不能在中断上下文中使用。

11.7.2 定时器竞争条件

因为定时器与当前执行代码是异步的，因此就有可能存在潜在的竞争条件。所以，首先，绝不能用如下所示的代码替代 `mod_timer()` 函数，来改变定时器的超时时间。这样的代码在多处理器机器上是不安全的：

```
del_timer(my_timer)
my_timer->expires = jiffies + new_delay;
add_timer(my_timer);
```

其次，一般情况下应该使用 `del_timer_sync()` 函数取代 `del_timer()` 函数，因为无法确定在删除定时器时，它是否正在其他处理器上运行。为了防止这种情况的发生，应该调用 `del_timer_sync()` 函数，而不是 `del_timer()` 函数。否则，对定时器执行删除操作后，代码会继续执行，但它有可能会去操作在其他处理器上运行的定时器正在使用的资源，因而造成并发访问，所以请优先使用删除定时器的同步方法。

最后，因为内核异步执行中断处理程序，所以应该重点保护定时器中断处理程序中的共享数据。定时器数据的保护问题曾在第 8 章和第 9 章讨论过。

11.7.3 实现定时器

内核在时钟中断发生后执行定时器，定时器作为软中断在下半部上下文中执行。具体来说，时钟中断处理程序会执行 `update_process_times()` 函数，该函数随即调用 `run_local_timers()` 函数：

```
void run_local_timers(void)
{
    hrtimer_run_queues();
    raise_softirq(TIMER_SOFTIRQ); /* 执行定时器软中断 */
    softlockup_tick();
}
```

`run_timer_softirq()` 函数处理软中断 `TIMER_SOFTIRQ`，从而在当前处理器上运行所有的（如果有的话）超时定时器。

虽然所有定时器都以链表形式存放在一起，但是让内核经常为了寻找超时定时器而遍历整个链表是不明智的。同样，将链表以超时时间进行排序也是很不明智的做法，因为这样一来在链表中插入和删除定时器都会很费时。为了提高搜索效率，内核将定时器按它们的超时时间划分为五组。当定时器超时时间接近时，定时器将随组一起下移。采用分组定时器的方法可以在执行软中断的多数情况下，确保内核尽可能减少搜索超时定时器所带来的负担。因此定时器管理代码是非常高效的。

11.8 延迟执行

内核代码（尤其是驱动程序）除了使用定时器或下半部机制以外，还需要其他方法来推迟执行任务。这种推迟通常发生在等待硬件完成某些工作时，而且等待的时间往往非常短，比如，重新设置网卡的以太模式需要花费 2ms，所以在设定网卡速度后，驱动程序必须至少等待 2ms 才能继续运行。

内核提供了许多延迟方法处理各种延迟要求。不同的方法有不同的处理特点，有些是在延迟任务时挂起处理器，防止处理器执行任何实际工作；另一些不会挂起处理器，所以也不能确保被延迟的代码能够在指定的延迟时间^①运行。

11.8.1 忙等待

最简单的延迟方法（虽然通常也是最不理想的办法）是忙等待（或者说忙循环）。但要注意该方法仅仅在想要延迟的时间是节拍的整数倍，或者精确率要求不高时才可以使用。

忙循环实现起来很简单——在循环中不断旋转直到希望的时钟节拍数耗尽，比如：

```
unsigned long timeout = jiffies+10;      /* 10 个节拍 */
while (time_before(jiffies, timeout))
    ;
```

循环不断执行，直到 jiffies 大于 delay 为止，总共的循环时间为 10 个节拍。在 HZ 值等于 1000 的 x86 体系结构上，耗时为 10ms。类似地：

```
unsigned long delay = jiffies + 2*HZ;    /* 2 秒 */
while(time_before(jiffies,delay))
    ;
```

程序要循环等待 2×HZ 个时钟节拍，也就是说无论时钟节拍率如何，都将等待 2s。

对于系统的其他部分，忙循环方法算不上一个好办法。因为当代码等待时，处理器只能在原地旋转等待——它不会去处理其他任何任务！事实上，你几乎不会用到这种低效率的办法，这里介绍它仅仅因为它是最简单最直接的延迟方法。当然你也可能在那些蹩脚的代码中发现它的身影。

更好的方法应该是在代码等待时，允许内核重新调度执行其他任务：

```
unsigned long delay = jiffies +5*HZ;
while(time_before(jiffies,delay))
    cond_resched();
```

cond_resched() 函数将调度一个新程序投入运行，但它只有在设置完 need_resched 标志后才能生效。换句话说，该方法有效的条件是系统中存在更重要的任务需要运行。注意，因为该方法需要调用调度程序，所以它不能在中断上下文中使用——只能在进程上下文中使用。事实上，所有延迟方法在进程上下文中使用得很好，因为中断处理程序都应该尽可能快地执行（忙循环与这种目标绝对是背道而驰）。另外，延迟执行不管在哪种情况下，都不应该在持有锁时或禁止中断时发生。

^① 事实上，没有方法能保证实际的延迟刚好等于指定的延迟时间，虽然可以非常接近，但是最精确的情况也只能达到接近，多数情况都要长于指定时间。

C 语言的推崇者可能会问：什么能保证前面的循环已经执行了。C 编译器通常只将变量装载一次。一般情况下不能保证循环中的 `jiffies` 变量在每次循环中被读取时都重新被载入。但是我们要求 `jiffies` 在每次循环时都必须重新装载，因为在后台 `jiffies` 值会随时钟中断的发生而不断增加。为了解决这个问题，`<linux/jiffies.h>` 中 `jiffies` 变量被标记为关键字 `volatile`。关键字 `volatile` 指示编译器在每次访问变量时都重新从主内存中获得，而不是通过寄存器中的变量别名来访问，从而确保前面的循环能按预期的方式执行。

11.8.2 短延迟

有时内核代码（通常也是驱动程序）不但需要很短暂的延迟（比时钟节拍还短），而且还要求延迟的时间很精确。这种情况多发生在和硬件同步时，也就是说需要短暂等待某个动作的完成（等待时间往往小于 1ms），所以不可能使用像前面例子中那种基于 `jiffies` 的延迟方法。对于频率为 100HZ 的时钟中断，它的节拍间隔甚至会超过 10ms！即使频率为 1000HZ 的时钟中断，节拍间隔也只能到 1ms，所以我们必须寻找其他方法满足更短、更精确的延迟要求。

幸运的是，内核提供了三个可以处理 ms、ns 和 μ s 级别的延迟函数，它们定义在文件 `<linux/delay.h>` 和 `<asm/delay.h>` 中，可以看到它们并不使用 `jiffies`：

```
void udelay(unsigned long usecs)
void ndelay(unsigned long nsecs)
void mdelay(unsigned long msecs)
```

前一个函数利用忙循环将任务延迟指定的 ms 数后运行，后者延迟指定的 μ s 数。众所周知，1s 等于 1000ms，等于 1000000 μ s。这个函数用起来很简单：

```
udelay(150);          /* 延迟 150  $\mu$ s */
```

`udelay()` 函数依靠执行数次循环达到延迟效果，而 `mdelay()` 函数又是通过 `udelay()` 函数实现的。因为内核知道处理器在 1 秒内能执行多少次循环（请看副栏中的 **BogoMIPS** 内容），所以 `udelay()` 函数仅仅需要根据指定的延迟时间在 1 秒中占的比例，就能决定需要进行多少次循环即可达到要求的推迟时间。

我的 BogoMIPS 比你的大

BogoMIPS 值总是让人觉得糊涂，也让人觉得有意思。其实，计算 BogoMIPS 并不是为了表现你的机器性能，它主要被 `udelay()` 函数和 `mdelay()` 函数使用。它的名字取自 `bogus`（也就是伪的）和 `MIPS`（每秒处理百万条指令）。大家都熟悉下面这样的系统启动信息（摘自一个装配主频为 2.4GHZ 的 7300 系列 Intel Xeon 处理器的机器启动信息）：

```
Detected 2400.131 MHz processor.
Calibrating delay loop ... 4799.56 BogoMIPS
```

BogoMIPS 值记录处理器在给定时间内忙循环执行的次数。其实，BogoMIPS 记录处理器在空闲时速度有多快。该值存放在变量 `loops_per_jiffy` 中，可以从文件 `/proc/cpuinfo` 中读到它。延迟循环函数使用 `loops_per_jiffy` 值来计算（相当准确）为提供精确延迟而需要进行多少次循环。

内核在启动时利用 `calibrate_delay()` 计算 `loops_per_jiffy` 值，该函数在文件 `init/main.c` 中。

`udelay()` 函数应当只在小延迟中调用，因为在快速机器上的大延迟可能导致溢出。通常，超过 1ms 的范围不要使用 `udelay()` 进行延迟。对于较长的延迟，`mdelay()` 工作良好。像其他忙等而延迟执行的方案，除非绝对必要，这两个函数（尤其是 `mdelay()`，因为用于长的延迟）都不应当使用。记住，持锁忙等或禁止中断是一种粗鲁的做法，因为系统响应时间和性能都会大受影响。不过，如果你需要精确的延迟，这些调用是最好的办法。这些忙等函数主要用在延迟小的地方，通常在 μs 范围内。

11.8.3 `schedule_timeout()`

更理想的延迟执行方法是使用 `schedule_timeout()` 函数，该方法会让需要延迟执行的任务睡眠到指定的延迟时间耗尽后再重新运行。但该方法也不能保证睡眠时间正好等于指定的延迟时间，只能尽量使睡眠时间接近指定的延迟时间。当指定的时间到期后，内核唤醒被延迟的任务并将其重新放回运行队列，用法如下：

```
/* 将任务设置为可中断睡眠状态 */
set_current_state(TASK_INTERRUPTIBLE);

/* 小睡一会儿，“s”秒后唤醒 */
schedule_timeout(s*HZ);
```

唯一的参数是延迟的相对时间，单位为 jiffies，上例中将相应的任务推入可中断睡眠队列，睡眠 s 秒。因为任务处于可中断状态，所以如果任务收到信号将被唤醒。如果睡眠任务不想接收信号，可以将任务状态设置为 `TASK_UNINTERRUPTIBLE`，然后睡眠。注意，在调用 `sechedule_timeout()` 函数前必须首先将任务设置成上面两种状态之一，否则任务不会睡眠。

注意，由于 `schedule_timeout()` 函数需要调用调度程序，所以调用它的代码必须保证能够睡眠（请参考第 8 章和第 9 章）。简而言之，调用代码必须处于进程上下文中，并且不能持有锁。

1. `schedule_timeout()` 的实现

`schedule_timeout()` 函数的用法相当简单、直接。其实，它是内核定时器的一个简单应用。请看下面的代码：

```
signed long schedule_timeout(signed long timeout)
{
    timer_t timer;
    unsigned long expire;

    switch (timeout)
    {
        case MAX_SCHEDULE_TIMEOUT:
            schedule();
            goto out;
        default:
            if (timeout < 0)
            {
                printk(KERN_ERR "schedule_timeout: wrong timeout "

```

```

        "value %lx from %p\n", timeout,
        __builtin_return_address(0));
    current->state = TASK_RUNNING;
    goto out;
}

}

expire = timeout + jiffies;

init_timer(&timer);
timer.expires = expire;
timer.data = (unsigned long) current;
timer.function = process_timeout;

add_timer(&timer);
schedule();
del_timer_sync(&timer);

timeout = expire - jiffies;

out:
    return timeout < 0 ? 0 : timeout;
}

```

该函数用原始的名字 `timer` 创建了一个定时器 `timer`；然后设置它的超时时间 `timeout`；设置超时执行函数 `process_timeout()`；接着激活定时器而且调用 `schedule()`。因为任务被标识为 `TASK_INTERRUPTIBLE` 或 `TASK_UNINTERRUPTIBLE`，所以调度程序不会再选择该任务投入运行，而会选择其他新任务运行。

当定时器超时时，`process_timeout()` 函数会被调用：

```

void process_timeout(unsigned long data)
{
    wake_up_process((task_t *)data);
}

```

该函数将任务设置为 `TASK_RUNNING` 状态，然后将其放入运行队列。

当任务重新被调度时，将返回代码进入睡眠前的位置继续执行（正好在调用 `schedule()` 后）。如果任务提前被唤醒（比如收到信号），那么定时器被撤销，`process_timeout()` 函数返回剩余的时间。

在 `switch()` 括号中的代码是为处理特殊情况而写的，正常情况不会用到它们。`MAX_SCHEDULE_TIMEOUT` 是用来检查任务是否无限期地睡眠，如果那样的话，函数不会为它设置定时器（因为睡眠时间没有期限），而这时调度程序会立刻被调用。如果你需要无限期地让任务睡眠，最好使用其他方法唤醒任务。

2. 设置超时时间，在等待队列上睡眠

第 4 章我们已经看到进程上下文中的代码为了等待特定事件发生，可以将自己放入等待队

列，然后调用调度程序去执行新任务。一旦事件发生后，内核调用 `wake_up()` 函数唤醒在睡眠队列上的任务，使其重新投入运行。

有时，等待队列上的某个任务可能既在等待一个特定事件到来，又在等待一个特定时间到期——就看谁来得更快。这种情况下，代码可以简单地使用 `schedule_timeout()` 函数代替 `schedule()` 函数，这样一来，当希望的指定时间到期，任务都会被唤醒。当然，代码需要检查被唤醒的原因（有可能是被事件唤醒，也有可能是因为延迟的时间到期，还可能是因为接收到了信号），然后执行相应的操作。

11.9 小结

在本章中，我们考察了时间的概念，并知道了墙上时钟与计算机的正常运行时间如何管理。我们对比了相对时间和绝对时间以及绝对事件与周期事件。我们还涵盖了诸如时钟中断、时钟节拍、HZ 以及 jiffies 等概念。

我们考察了定时器的实现，了解了如何把这些用到自己的内核代码中。本章最后，我们浏览了开发者用于延迟的其他方法。

你写的大多数内核代码都需要对时间及其走过的时间有一些理解。而最大的可能是，只要你编写驱动程序，就需要处理内核定时器。与其让时间悄悄溜走，还不如阅读本章。



第 12 章

内存管理

在内核里分配内存可不像在其他地方分配内存那么容易。造成这种局面的因素很多。从根本上来讲，是因为内核本身不能像用户空间那样奢侈地使用内存。内核与用户空间不同，它不具备这种能力，它不支持简单便捷的内存分配方式。比如，内核一般不能睡眠。此外，处理内存分配错误对内核来说也绝非易事。正是由于这些限制，再加上内存分配机制不能太复杂，所以在内核中获取内存要比在用户空间复杂得多。不过，从程序开发者角度来看，也不是说内核的内存分配就困难得不得了，只是和用户空间中的内存分配不太一样而已。

本章讨论的是在内核之中获取内存的方法。在深入研究实际的分配接口之前，我们需要理解内核是如何管理内存的。

12.1 页

内核把物理页作为内存管理的基本单位。尽管处理器的最小可寻址单位通常为字（甚至字节），但是，内存管理单元（MMU，管理内存并把虚拟地址转换为物理地址的硬件）通常以页为单位进行处理。正因为如此，MMU 以页（page）大小为单位来管理系统中的页表（这也是页表名的来由）。从虚拟内存的角度来看，页就是最小单位。

在第 19 章中我们将会看到，体系结构不同，支持的页大小也不尽相同，还有些体系结构甚至支持几种不同的页大小。大多数 32 位体系结构支持 4KB 的页，而 64 位体系结构一般会支持 8KB 的页。这就意味着，在支持 4KB 页大小并有 1GB 物理内存的机器上，物理内存会被划分为 262144 个页。

内核用 `struct page` 结构表示系统中的每个物理页，该结构位于 `<linux/mm_types.h` 中——我简化了定义，去除了两个容易混淆我们讨论主题的联合结构体：

```
struct page {
    unsigned long    flags;
    atomic_t         _count;
    atomic_t         _mapcount;
    unsigned long    private;
    struct address_space *mapping;
    pgoff_t          index;
    struct list_head lru;
    void             *virtual;
};
```

让我们看一下其中比较重要的域。`flag` 域用来存放页的状态。这些状态包括页是不是脏的，

是不是被锁定在内存中等。flag 的每一位单独表示一种状态，所以它至少可以同时表示出 32 种不同的状态。这些标志定义在 `<linux/page-flags.h>` 中。

`_count` 域存放页的引用计数——也就是这一页被引用了多少次。当计数值变为 -1 时，就说明当前内核并没有引用这一页，于是，在新的分配中就可以使用它。内核代码不应当直接检查该域，而是调用 `page_count()` 函数进行检查，该函数唯一的参数就是 `page` 结构。当页空闲时，尽管该结构内部的 `_count` 值是负的，但是对 `page_count()` 函数而言，返回 0 表示页空闲，返回一个正整数表示页在使用。一个页可以由页缓存使用（这时，`mapping` 域指向和这个页关联的 `addresss_space` 对象），或者作为私有数据（由 `private` 指向），或者作为进程页表中的映射。

`virtual` 域是页的虚拟地址。通常情况下，它就是页在虚拟内存中的地址。有些内存（即所谓的高端内存）并不永久地映射到内核地址空间上。在这种情况下，这个域的值是 `NULL`，需要的时候，必须动态地映射这些页。稍后我们将讨论高端内存。

必须要理解的一点是 `page` 结构与物理页相关，而并非与虚拟页相关。因此，该结构对页的描述只是短暂的。即使页中所包含的数据继续存在，由于交换等原因，它们也可能不再和同一个 `page` 结构相关联。内核仅仅用这个数据结构来描述当前时刻在相关的物理页中存放的东西。这种数据结构的目的在于描述物理内存本身，而不是描述包含在其中的数据。

内核用这一结构来管理系统中所有的页，因为内核需要知道一个页是否空闲（也就是页有没有被分配）。如果页已经被分配，内核还需要知道谁拥有这个页。拥有者可能是用户空间进程、动态分配的内核数据、静态内核代码或页高速缓存等。

系统中的每个物理页都要分配一个这样的结构体，开发者常常对此感到惊讶。他们会想“这得浪费多少内存呀”！让我们来算算对所有这些页都这么做，到底要消耗掉多少内存。就算 `struct page` 占 40 字节的内存吧，假定系统的物理页为 8KB 大小，系统有 4GB 物理内存。那么，系统中共有页面 524 288 个，而描述这么多页面的 `page` 结构体消耗的内存只不过是 20MB；也许绝对值不小，但是相对系统 4GB 内存而言，仅是很小的一部分罢了。因此，要管理系统中这么多物理页面，这个代价并不算太高。

12.2 区

由于硬件的限制，内核并不能对所有的页一视同仁。有些页位于内存中特定的物理地址上，所以不能将其用于一些特定的任务。由于存在这种限制，所以内核把页划分为不同的区（`zone`）。内核使用区对具有相似特性的页进行分组。Linux 必须处理如下两种由于硬件存在缺陷而引起的内存寻址问题：

- 一些硬件只能用某些特定的内存地址来执行 DMA（直接内存访问）。
- 一些体系结构的内存的物理寻址范围比虚拟寻址范围大得多。这样，就有一些内存不能永久地映射到内核空间上。

因为存在这些制约条件，Linux 主要使用了四种区：

- `ZONE_DMA`——这个区包含的页能用来执行 DMA 操作。
- `ZONE_DMA32`——和 `ZONE_DMA` 类似，该区包含的页面可用来执行 DMA 操作；而和 `ZONE_DMA` 不同之处在于，这些页面只能被 32 位设备访问。在某些体系结构中，该区将

比 ZONE_DMA 更大。

- ZONE_NORMAL——这个区包含的都是能正常映射的页。
- ZONE_HIGHMEM——这个区包含“高端内存”，其中的页并不能永久地映射到内核地址空间。这些区（还有两种不大重要的）在 <linux/mmzone.h> 中定义。

区的实际使用和分布是与体系结构相关的。例如，某些体系结构在内存的任何地址上执行 DMA 都没有问题。在这些体系结构中，ZONE_DMA 为空，ZONE_NORMAL 就可以直接用于分配。与此相反，在 x86 体系结构上，ISA 设备就不能在整个 32 位^①的地址空间中执行 DMA，因为 ISA 设备只能访问物理内存的前 16MB。因此，ZONE_DMA 在 x86 上包含的页都在 0-16MB 的内存范围里。

ZONE_HIGHMEM 的工作方式也差不多。能否直接映射取决于体系结构。在 32 位 x86 系统上，ZONE_HIGHMEM 为高于 896MB 的所有物理内存。在其他体系结构上，由于所有内存都被直接映射，所以 ZONE_HIGHMEM 为空。ZONE_HIGHMEM 所在的内存就是所谓的高端内存^②（high memory）。系统的其余内存就是所谓的低端内存（low memory）。

前两个区各取所需之后，剩余的就由 ZONE_NORMAL 区独享了。在 x86 上，ZONE_NORMAL 是从 16MB 到 896MB 的所有物理内存。在其他（更幸运）的体系结构上，ZONE_NORMAL 是所有的可用物理内存。表 12-1 是每个区及其在 x86-32 上所占页的列表。

表 12-1 x86-32 上的区

区	描 述	物 理 内 存
ZONE_DMA	DMA 使用的页	<16MB
ZONE_NORMAL	正常可寻址的页	16 ~ 896MB
ZONE_HIGHMEM	动态映射的页	>896MB

Linux 把系统的页划分为区，形成不同的内存池，这样就可以根据用途进行分配了。例如，ZONE_DMA 内存池让内核有能力为 DMA 分配所需的内存。如果需要这样的内存，那么，内核就可以从 ZONE_DMA 中按照请求的数目取出页。注意，区的划分没有任何物理意义，这只不过是内核为了管理页而采取的一种逻辑上的分组。

某些分配可能需要从特定的区中获取页，而另外一些分配则可以从多个区中获取页。比如，尽管用于 DMA 的内存必须从 ZONE_DMA 中进行分配，但是一般用途的内存却既能从 ZONE_DMA 分配，也能从 ZONE_NORMAL 分配，不过不可能同时从两个区分配，因为分配是不能跨区界限的。当然，内核更希望一般用途的内存从常规区分配，这样能节省 ZONE_DMA 中的页，保证满足 DMA 的使用需求。但是，如果可供分配的资源不够用了（如果内存已经变得很少了），那么，内核就会去占用其他可用区的内存。

不是所有的体系结构都定义了全部区，有些 64 位的体系结构，如 Intel 的 x86-64 体系结构

① 有些糟糕的 PCI 设备只能在 24 位地址空间内执行 DMA 操作。

② Linux 的高端内存和 DOS 的高端内存没有关系，DOS 的高端内存是围绕 DOS 和 x86 的“实模式”的空间范围限制而言的。

可以映射和处理 64 位的内存空间，所以 x86-64 没有 ZONE_HIGHMEM 区，所有的物理内存都处于 ZONE_DMA 和 ZONE_NORMAL 区。

每个区都用 struct zone 表示，在 <linux/mmzone.h> 中定义：

```
struct zone {
    unsigned long          watermark[NR_WMARK];
    unsigned long          lowmem_reserve[MAX_NR_ZONES];
    struct per_cpu_pageset pageset[NR_CPUS];
    spinlock_t             lock;
    struct free_area        free_area[MAX_ORDER]
    spinlock_t             lru_lock;
    struct zone_lru {
        struct list_head list;
        unsigned long      nr_saved_scan;
    } lru[NR_LRU_LISTS];
    struct zone_reclaim_stat reclaim_stat;
    unsigned long          pages_scanned;
    unsigned long          flags;
    atomic_long_t          vm_stat[NR_VM_ZONE_STAT_ITEMS];
    int                    prev_priority;
    unsigned               int inactive_ratio;
    wait_queue_head_t      *wait_table;
    unsigned long          wait_table_hash_nr_entries;
    unsigned long          wait_table_bits;
    struct pglist_data      *zone_pgdat;
    unsigned long          zone_start_pfn;
    unsigned long          spanned_pages;
    unsigned long          present_pages;
    const char             *name;
};
```

这个结构体很大，但是，系统中只有三个区，因此，也只有三个这样的结构。让我们看一下其中一些重要的域。

lock 域是一个自旋锁，它防止该结构被并发访问。注意，这个域只保护结构，而不保护驻留在这个区中的所有页。没有特定的锁来保护单个页，但是，部分内核可以锁住在页中驻留的数据。

watermark 数组持有该区的最小值、最低和最高水位值。内核使用水位为每个内存区设置合适的内存消耗基准。该水位随空闲内存的多少而变化。

name 域是一个以 NULL 结束的字符串表示这个区的名字。内核启动期间初始化这个值，其代码位于 mm/page_alloc.c 中。三个区的名字分别为“DMA”、“Normal”和“HighMem”。

12.3 获得页

我们已经对内核如何管理内存（页、区等）有所了解了，现在让我们看一下内核实现的接口，我们正是通过这些接口在内核内分配和释放内存的。

内核提供了一种请求内存的底层机制，并提供了对它进行访问的几个接口。所有这些接口都

以页为单位分配内存，定义于 <linux/gfp.h> 中。最核心的函数是：

```
struct page * alloc_pages(gfp_t gfp_mask, unsigned int order)
```

该函数分配 2^{order} ($1 \leq \text{order}$) 个连续的物理页，并返回一个指针，该指针指向第一个页的 page 结构体；如果出错，就返回 NULL。在 12.4 节我们再研究 gfp_t 类型和 gfp_mask 参数。你可以用下面这个函数把给定的页转换成它的逻辑地址：

```
void * page_address(struct page *page)
```

该函数返回一个指针，指向给定物理页当前所在的逻辑地址。如果你无须用到 struct page，你可以调用：

```
unsigned long __get_free_pages(gfp_t gfp_mask, unsigned int order)
```

这个函数与 alloc_pages() 作用相同，不过它直接返回所请求的第一个页的逻辑地址。因为页是连续的，所以其他页也会紧随其后。

如果你只需一页，就可以用下面两个封装好的函数，它能让你少敲几下键盘：

```
struct page * alloc_page(gfp_t gfp_mask)
unsigned long __get_free_page(gfp_t gfp_mask)
```

这两个函数与其兄弟函数工作方式相同，只不过传递给 order 的值为 0 ($2^0 = 1$ 页)。

12.3.1 获得填充为 0 的页

如果你需要让返回的页的内容全为 0，请用下面这个函数：

```
unsigned long get_zeroed_page(unsigned int gfp_mask)
```

这个函数与 __get_free_pages() 工作方式相同，只不过把分配好的页都填充成了 0——字节中的每一位都要取消设置。如果分配的页是给用户空间的，这个函数就非常有用了。虽说分配好的页中应该包含的都是随机产生的垃圾信息，但其实这些信息可能并不是完全随机的——它很可能“随机地”包含某些敏感数据。用户空间的页在返回之前，所有数据必须填充为 0，或做其他清理工作，在保障系统安全这一点上，我们决不妥协。表 12-2 是所有底层的页分配方法的列表。

表 12-2 低级页分配方法

标 志	描 述
alloc_page(gfp_mask)	只分配一页，返回指向页结构的指针
alloc_pages(gfp_mask, order)	分配 2^{order} 个页，返回指向第一页页结构的指针
__get_free_page(gfp_mask)	只分配一页，返回指向其逻辑地址的指针
__get_free_pages(gfp_mask, order)	分配 2^{order} 页，返回指向第一页逻辑地址的指针
get_zeroed_page(gfp_mask)	只分配一页，让其内容填充 0，返回指向其逻辑地址的指针

12.3.2 释放页

当你不再需要页时可以用下面的函数释放它们：

```
void __free_pages(struct page *page, unsigned int order)
void free_pages(unsigned long addr, unsigned int order)
void free_page(unsigned long addr)
```

释放页时要谨慎，只能释放属于你的页。传递了错误的 `struct page` 或地址，用了错误的 `order` 值，这些都可能系统崩溃。请记住，内核是完全信赖自己的。这点与用户空间不同，如果你有非法操作，内核会开开心心地把自己挂起来，停止运行。

让我们看一个例子。其中，我们想得到 8 个页：

```
unsigned long page;

page = __get_free_pages(GFP_KERNEL, 3);
if (!page){
    /* 没有足够的内存：你必须处理这种错误！ */
    return -ENOMEM;
}
/* “page” 现在指向 8 个连续页中第 1 个页的地址 ... */
```

在此，我们使用完这 8 个页之后释放它们：

```
free_pages(page, 3);

/*
 * 页现在已经被释放了，我们不应该再访问
 * 存放在 “page” 中的地址了
 */
```

`GFP_KERNEL` 参数是 `gfp_mask` 标志的一个例子。前面我们已经简要讨论过。

调用 `__get_free_pages()` 之后要注意进行错误检查。内核分配可能失败，因此你的代码必须进行检查并做相应的处理。这意味在此之前，你所做的所有工作可能前功尽弃，甚至还需要回归到原来的状态。正因为如此，在程序开始时就先进行内存分配是很有意义的，这能让错误处理得容易一点。如果你不这么做，那么在你想要分配内存的时候如果失败了，局面可能就难以控制了。

当你需要以页为单位的一族连续物理页时，尤其是在你只需要一两页时，这些低级页函数很有用。对于常用的以字节为单位的分配来说，内核提供的函数是 `kmalloc()`。

12.4 kmalloc()

`kmalloc()` 函数与用户空间的 `malloc()` 一族函数非常类似，只不过它多了一个 `flags` 参数。`kmalloc()` 函数是一个简单的接口，用它可以获得以字节为单位的一块内核内存。如果你需要整个页，那么，前面讨论的页分配接口可能是更好的选择。但是，对于大多数内核分配来说，`kmalloc()` 接口用得更多。

`kmalloc()` 在 `<linux/slab.h>` 中声明：

```
void * kmalloc(size_t size, gfp_t flags)
```

这个函数返回一个指向内存块的指针，其内存块至少要有 size 大小。所分配的内存区在物理上是连续的。在出错时，它返回 NULL。除非没有足够的内存可用，否则内核总能分配成功。在对 kmalloc() 调用之后，你必须检查返回的是不是 NULL，如果是，要适当地处理错误。

让我们看一个例子。我们随便假定存在一个 dog 结构体，现在需要为它动态地分配足够的空间：

```
struct dog *p;

p = kmalloc(sizeof(struct dog), GFP_KERNEL);
if (!p)
    /* 处理错误 ... */
```

如果 kmalloc() 调用成功，那么，ptr 现在指向一个内存块，内存块的大小至少为所请求的大小。GFP_KERNEL 标志表示在试图获取内存并返回给 kmalloc() 的调用者的过程中，内存分配器将要采取的行为。

12.4.1 gfp_mask 标志

我们已经看过了几个例子，发现不管是在低级页分配函数中，还是在 kmalloc() 中，都用到了分配器标志。现在，我们就深入讨论一下这些标志。

这些标志可分为三类：行为修饰符、区修饰符及类型。行为修饰符表示内核应当如何分配所需的内存。在某些特定情况下，只能使用某些特定的方法分配内存。例如，中断处理程序就要求内核在分配内存的过程中不能睡眠（因为中断处理程序不能被重新调度）。区修饰符表示从哪儿分配内存。前面我们已经看到，内核把物理内存分为多个区，每个区用于不同的目的。区修饰符指明到底从这些区中的哪一区中进行分配。类型标志组合了行为修饰符和区修饰符，将各种可能用到的组合归纳为不同类型，简化了修饰符的使用；这样，你只需指定一个类型标志就可以了。GFP_KERNEL 就是一种类型标志，内核中进程上下文相关的代码可以使用它。我们来看一下这些标志。

1. 行为修饰符

所有这些标志，包括行为描述符都是在 <linux/gfp.h> 中声明的。不过，在 <linux/slab.h> 中包含有这个头文件，因此，你一般不必直接包含引用它。实际上，一般只使用类型修饰符就够了，我们随后会看到这点。因此，最好对每个标志都有所了解。表 12-3 是行为修饰符的列表。

表 12-3 行为修饰符

标 志	描 述
__GFP_WAIT	分配器可以睡眠
__GFP_HIGH	分配器可以访问紧急事件缓冲池
__GFP_IO	分配器可以启动磁盘 I/O
__GFP_FS	分配器可以启动文件系统 I/O
__GFP_COLD	分配器应该使用高速缓存中快要淘汰出去的页
__GFP_NOWARN	分配器将不打印失败警告
__GFP_REPEAT	分配器在分配失败时重复进行分配，但是这次分配还存在失败的可能
__GFP_NOFAIL	分配器将无限地重复进行分配。分配不能失败

(续)

标 志	描 述
<code>__GFP_NORETRY</code>	分配器在分配失败时绝不会重新分配
<code>__GFP_NO_GROW</code>	由 slab 层内部使用
<code>__GFP_COMP</code>	添加混合页元数据，在 hugetlb 的代码内部使用

可以同时指定这些分配标志。例如：

```
ptr = kmalloc(size, __GFP_WAIT | __GFP_IO | __GFP_FS);
```

说明页分配器（最终调用 `alloc_pages()`）在分配时可以阻塞、执行 I/O，在必要时还可以执行文件系统操作。这就让内核有很大的自由度，以便它尽可能找到空闲的内存来满足分配请求。

大多数分配都会指定这些修饰符，但一般不是这样直接指定，而是采用我们随后讨论的类型标志。别担心，你不会在分配内存时为怎样使用这些标志而犯愁的！

2. 区修饰符

区修饰符表示内存区应当从何处分配。通常，分配可以从任何区开始。不过，内核优先从 `ZONE_NORMAL` 开始，这样可以确保其他区在需要时有足够的空闲页可供使用。

实际上只有两个区修饰符，因为除了 `ZONE_NORMAL` 之外只有两个区（默认都是从 `ZONE_NORMAL` 区进行分配）。表 12-4 是区修饰符的列表。

表 12-4 区修饰符

标 志	描 述
<code>__GFP_DMA</code>	从 <code>ZONE_DMA</code> 分配
<code>__GFP_DMA32</code>	只在 <code>ZONE_DMA32</code> 分配
<code>__GFP_HIGHMEM</code>	从 <code>ZONE_HIGHMEM</code> 或 <code>ZONE_NORMAL</code> 分配

指定以上标志中的一个就可以改变内核试图进行分配的区。`__GFP_DMA` 标志强制内核从 `ZONE_DMA` 分配。这个标志在说，有了这种奇怪的标识，我绝对可以拥有进行 DMA 的内存。相反，如果指定 `__GFP_HIGHMEM` 标志，则从 `ZONE_HIGHMEM`（优先）或 `ZONE_NORMAL` 分配。这个标志在说，我可以使用高端内存，因此，我可以是一个玩偶，给你退还一些内存，但是，常规内存还照常工作。如果没有指定任何标志，则内核从 `ZONE_DMA` 或 `ZONE_NORMAL` 进行分配，当然优先从 `ZONE_NORMAL` 进行分配。不管区标志说什么了，只要它行为正常，我就不关心了。

不能给 `_get_free_pages()` 或 `kalloc()` 指定 `ZONE_HIGHMEM`，因为这两个函数返回的都是逻辑地址，而不是 page 结构，这两个函数分配的内存当前有可能还没有映射到内核的虚拟地址空间，因此，也可能根本就没有逻辑地址。只有 `alloc_pages()` 才能分配高端内存。实际上，你的分配在大多数情况下都不必指定修饰符，`ZONE_NORMAL` 就足矣。

3. 类型标志

类型标志指定所需的行为和区描述符以完成特殊类型的处理。正因为这一点，内核代码趋向

于使用正确的类型标志，而不是一味地指定它可能需要用到的多个描述符。这么做既简单又不容易出错误。表 12-5 是类型标志的列表，而表 12-6 显示了每个类型标志与哪些修饰符相关联。

表 12-5 类型标志

标 志	描 述
GFP_ATOMIC	这个标志用在中断处理程序、下半部、持有自旋锁以及其他不能睡眠的地方
GFP_NOWAIT	与 GFP_ATOMIC 类似，不同之处在于，调用不会退给紧急内存池。这就增加了内存分配失败的可能性。
GFP_NOIO	这种分配可以阻塞，但不会启动磁盘 I/O。这个标志在不能引发更多磁盘 I/O 时能阻塞 I/O 代码，这可能导致令人不愉快的递归
GFP_NOFS	这种分配在必要时可能阻塞，也可能启动磁盘 I/O，但是不会启动文件系统操作。这个标志在你不能再启动另一个文件系统的操作时，用在文件系统部分的代码中
GFP_KERNEL	这是一种常规分配方式，可能会阻塞。这个标志在睡眠安全时用在进程上下文代码中。为了获得调用者所需的内存，内核会尽力而为。这个标志应当是首选标志
GFP_USER	这是一种常规分配方式，可能会阻塞。这个标志用于为用户空间进程分配内存时
GFP_HIGHUSER	这是从 ZONE_HIGHMEM 进行分配，可能会阻塞。这个标志用于为用户空间进程分配内存
GFP_DMA	这是从 ZONE_DMA 进行分配。需要获取能供 DMA 使用的内存的设备驱动程序使用这个标志，通常与以上的某个标志组合在一起使用

表 12-6 在每种类型标志后隐含的修饰符列表

标 志	修饰符标志
GFP_ATOMIC	__GFP_HIGH
GFP_NOWAIT	0
GFP_NOIO	__GFP_WAIT
GFP_NOFS	(__GFP_WAIT __GFP_IO)
GFP_KERNEL	(__GFP_WAIT __GFP_IO __GFP_FS)
GFP_USER	(__GFP_WAIT __GFP_IO __GFP_FS)
GFP_HIGHUSER	(__GFP_WAIT __GFP_IO __GFP_FS __GFP_HIGHMEM)
GFP_DMA	__GFP_DMA

让我们看一下最常用的标志以及你什么时候、为什么需要使用它们。内核中最常用的标志是 GFP_KERNEL。这种分配可能会引起睡眠，它使用的是普通优先级。因为调用可能阻塞，因此这个标志只用在可以重新安全调度的进程上下文中（也就是没有锁被持有等情况）。因为这个标志对内核如何获取请求的内存没有任何约束，所以内存分配成功的可能性很高。

另一个截然相反的标志是 GFP_ATOMIC。因为这个标志表示不能睡眠的内存分配，因此想要满足调用者获取内存的请求将会受到很严格的限制。即使没有足够的连续内存块可供使用，内核也很可能无法释放出可用内存来，因为内核不能让调用者睡眠。相反，GFP_KERNEL 分配可以让调用者睡眠、交换、刷新一些页到硬盘等。因为 GFP_ATOMIC 不能执行以上任何操作，因此与 GFP_KERNEL 相比较，它分配成功的机会较小（尤其在内存短缺时）。即便如此，在当前代码（例如中断处理程序、软中断和 tasklet）不能睡眠时，也只能选择 GFP_ATOMIC。

在以上两种标志中间的是 GFP_NOIO 和 GFP_NOFS。以这两个标志进行的分配可能会引起阻塞，但它们会避免执行某些其他操作。GFP_NOIO 分配绝不会启动任何磁盘 I/O 来帮助满足

请求。而 GFP_NOFS 可能会启动磁盘 I/O，但是它不会启动文件系统 I/O。你为什么需要这些标志？它们分别用在某些低级块 I/O 或文件系统的代码中。设想，如果文件系统代码中需要分配内存，但没有使用 GFP_NOFS。这种分配可能会引起更多的文件系统操作，而这些操作又会导致另外的分配，从而再引起更多的文件系统操作！这会一直持续下去。这样的代码在调用分配器的时候，必须确保分配器不会再执行到代码本身，否则，分配就可能产生死锁。也别紧张，内核使用这两个标志的地方是极少的。

GFP_DMA 标志表示分配器必须满足从 ZONE_DMA 进行分配的请求。这个标志用在需要 DMA 的内存的设备驱动程序中。一般你会把这个标志与 GFP_ATOMIC 和 GFP_KERNEL 结合起来使用。

在你编写的绝大多数代码中，用到的要么是 GFP_KERNEL，要么是 GFP_ATOMIC。表 12-7 是通常情形和所用标志的列表。不管使用哪种分配类型，你都必须进行检查，并对错误进行处理。

表 12-7 什么时候用哪种标志

情 形	相应标志
进程上下文，可以睡眠	使用 GFP_KERNEL
进程上下文，不可以睡眠	使用 GFP_ATOMIC，在你睡眠之前或之后以 GFP_KERNEL 执行内存分配
中断处理程序	使用 GFP_ATOMIC
软中断	使用 GFP_ATOMIC
tasklet	使用 GFP_ATOMIC
需要用于 DMA 的内存，可以睡眠	使用 (GFP_DMA GFP_KERNEL)
需要用于 DMA 的内存，不可以睡眠	使用 (GFP_DMA GFP_ATOMIC)，或在你睡眠之前执行内存分配

12.4.2 kfree()

kmalloc() 的另一端就是 kfree()，kfree() 声明于 <linux/slab.h> 中：

```
void kfree(const void *ptr)
```

kfree() 函数释放由 kmalloc() 分配出来的内存块。如果想要释放的内存不是由 kmalloc() 分配的，或者想要释放的内存早就被释放了，比如说释放属于内核其他部分的内存，调用这个函数就会导致严重的后果。与用户空间类似，分配和回收要注意配对使用，以避免内存泄漏和其他 bug。注意，调用 kfree(NULL) 是安全的。

让我们看一个在中断处理程序中分配内存的例子。在这个例子中，中断处理程序想分配一个缓冲区来保存输入数据。BUF_SIZE 预定义为以字节为单位的缓冲区长度，它应该是大于两个字节的。

```
char *buf;

buf = kmalloc(BUF_SIZE, GFP_ATOMIC);
if (!buf)
    /* 内存分配出错！ */
```

之后，当我们不再需要这个内存时，别忘了释放它：

```
kfree(buf);
```

12.5 vmalloc()

`vmalloc()` 函数的工作方式类似于 `kmalloc()`，只不过前者分配的内存虚拟地址是连续的，而物理地址则无须连续。这也是用户空间分配函数的工作方式：由 `malloc()` 返回的页在进程的虚拟地址空间内是连续的，但是，这并不保证它们在物理 RAM 中也是连续的。`kmalloc()` 函数确保页在物理地址上是连续的（虚拟地址自然也是连续的）。`vmalloc()` 函数只确保页在虚拟地址空间内是连续的。它通过分配非连续的物理内存块，再“修正”页表，把内存映射到逻辑地址空间的连续区域中，就能做到这点。

大多数情况下，只有硬件设备需要得到物理地址连续的内存。在很多体系结构上，硬件设备存在于内存管理单元以外，它根本不理解什么是虚拟地址。因此，硬件设备用到的任何内存区都必须是物理上连续的块，而不仅仅是虚拟地址连续上的块。而仅供软件使用的内存块（例如与进程相关的缓冲区）就可以使用只有虚拟地址连续的内存块。但在你的编程中，根本察觉不到这种差异。对内核而言，所有内存看起来都是逻辑上连续的。

尽管在某些情况下才需要物理上连续的内存块，但是，很多内核代码都用 `kmalloc()` 来获得内存，而不是 `vmalloc()`。这主要是出于性能的考虑。`vmalloc()` 函数为了把物理上不连续的页转换为虚拟地址空间上连续的页，必须专门建立页表项。糟糕的是，通过 `vmalloc()` 获得的页必须一个一个地进行映射（因为它们物理上是不连续的），这就会导致比直接内存映射大得多的 TLB[⊖] 抖动。因为这些原因，`vmalloc()` 仅在不得已时才会使用——典型的就是为了获得大块内存。时，例如，当模块被动态插入到内核中时，就把模块装载到由 `vmalloc()` 分配的内存上。

`vmalloc()` 函数声明在 `<linux/vmalloc.h>` 中，定义在 `<mm/vmalloc.c>` 中。用法与用户空间的 `malloc()` 相同：

```
void * vmalloc(unsigned long size)
```

该函数返回一个指针，指向逻辑上连续的一块内存区，其大小至少为 `size`。在发生错误时，函数返回 `NULL`。函数可能睡眠，因此，不能从中断上下文中进行调用，也不能从其他不允许阻塞的情况下进行调用。

要释放通过 `vmalloc()` 所获得的内存，使用下面的函数：

```
void vfree(const void *addr)
```

这个函数会释放从 `addr` 开始的内存块，其中 `addr` 是以前由 `vmalloc()` 分配的内存块的地址。这个函数也可以睡眠，因此，不能从中断上下文中调用。它没有返回值。

这个函数用起来比较简单：

⊖ TLB (translation lookaside buffer) 是一种硬缓冲区，很多体系结构用它来缓存虚拟地址到物理地址的映射关系。它极大地提高了系统的性能，因为大多数内存都要进行虚拟寻址。

```
char *buf;

buf = vmalloc(16 * PAGE_SIZE); /* get 16 pages */
if (!buf)
    /* 错误！不能分配内存 */
/*
 * buf 现在指向虚拟地址连续的一块内存区，其大小至少为 16*PAGE_SIZE
 */
```

在分配内存之后，一定要释放它：

```
vfree(buf);
```

12.6 slab 层

分配和释放数据结构是所有内核中最普遍的操作之一。为了便于数据的频繁分配和回收，编程人员常常会用到空闲链表。空闲链表包含可供使用的、已经分配好的数据结构块。当代码需要一个新的数据结构实例时，就可以从空闲链表中抓取一个，而不需要分配内存，再把数据放进去。以后，当不再需要这个数据结构的实例时，就把它放回空闲链表，而不是释放它。从这个意义上说，空闲链表相当于对象高速缓存——快速存储频繁使用的对象类型。

在内核中，空闲链表面临的主要问题之一是不能全局控制。当可用内存变得紧缺时，内核无法通知每个空闲链表，让其收缩缓存的大小以便释放出一些内存来。实际上，内核根本就不知道存在任何空闲链表。为了弥补这一缺陷，也为了使代码更加稳固，Linux 内核提供了 slab 层（也就是所谓的 slab 分配器）。slab 分配器扮演了通用数据结构缓存层的角色。

slab 分配器的概念首先在 Sun 公司的 SunOS 5.4 操作系统中得以实现^①。Linux 数据结构缓存层具有同样的名字和基本设计思想。

slab 分配器试图在几个基本原则之间寻求一种平衡：

- 频繁使用的数据结构也会频繁分配和释放，因此应当缓存它们。
- 频繁分配和回收必然会导致内存碎片（难以找到大块连续的可用内存）。为了避免这种现象，空闲链表的缓存会连续地存放。因为已释放的数据结构又会放回空闲链表，因此不会导致碎片。
- 回收的对象可以立即投入下一次分配，因此，对于频繁的分配和释放，空闲链表能够提高其性能。
- 如果分配器知道对象大小、页大小和总的高速缓存的大小这样的概念，它会做出更明智的决策。
- 如果让部分缓存专属于单个处理器（对系统上的每个处理器独立而唯一），那么，分配和释放就可以在不加 SMP 锁的情况下进行。
- 如果分配器是与 NUMA 相关的，它就可以从相同的内存节点为请求者进行分配。
- 对存放的对象进行着色（color），以防止多个对象映射到相同的高速缓存行（cache line）。

Linux 的 slab 层在设计和实现时充分考虑了上述原则。

^① 参看 J. Bonwick 所著的《The Slab Allocator : An Object-Caching Kernel Memory Allocator》，USENIX，1994。

12.6.1 slab 层的设计

slab 层把不同的对象划分为所谓高速缓存组，其中每个高速缓存组都存放不同类型的对象。每种对象类型对应一个高速缓存。例如，一个高速缓存用于存放进程描述符（task_struct 结构的一个空闲链表），而另一个高速缓存存放索引节点对象（struct inode）。有趣的是，kmalloc() 接口建立在 slab 层之上，使用了一组通用高速缓存。

然后，这些高速缓存又被划分为 slab（这也是这个子系统名字的来由）。slab 由一个或多个物理上连续的页组成。一般情况下，slab 也就仅仅由一页组成。每个高速缓存可以由多个 slab 组成。

每个 slab 都包含一些对象成员，这里的对象指的是被缓存的数据结构。每个 slab 处于三种状态之一：满、部分满或空。一个满的 slab 没有空闲的对象（slab 中的所有对象都已被分配）。一个空的 slab 没有分配出任何对象（slab 中的所有对象都是空闲的）。一个部分满的 slab 有一些对象已分配出去，有些对象还空闲着。当内核的某一部分需要一个新的对象时，先从部分满的 slab 中进行分配。如果没有部分满的 slab，就从空的 slab 中进行分配。如果没有空的 slab，就要创建一个 slab 了。显然，满的 slab 无法满足请求，因为它根本就没有空闲的对象。这种策略能减少碎片。

作为一个例子，让我们考察一下 inode 结构，该结构是磁盘索引节点在内存中的体现（参见第 13 章）。这些数据结构会频繁地创建和释放，因此，用 slab 分配器来管理它们就很有必要。因而 struct inode 就由 inode_cachep 高速缓存（这是一种标准的命名规范）进行分配。这种高速缓存由一个或多个 slab 组成——由多个 slab 组成的可能性大一些，因为这样的对象数量很大。每个 slab 包含尽可能多的 struct inode 对象。当内核请求分配一个新的 inode 结构时，内核就从部分满的 slab 或空的 slab（如果没有部分满的 slab）返回一个指向已分配但未使用的结构的指针。当内核用完 inode 对象后，slab 分配器就把该对象标记为空闲。图 12-1 显示了高速缓存、slab 及对象之间的关系。

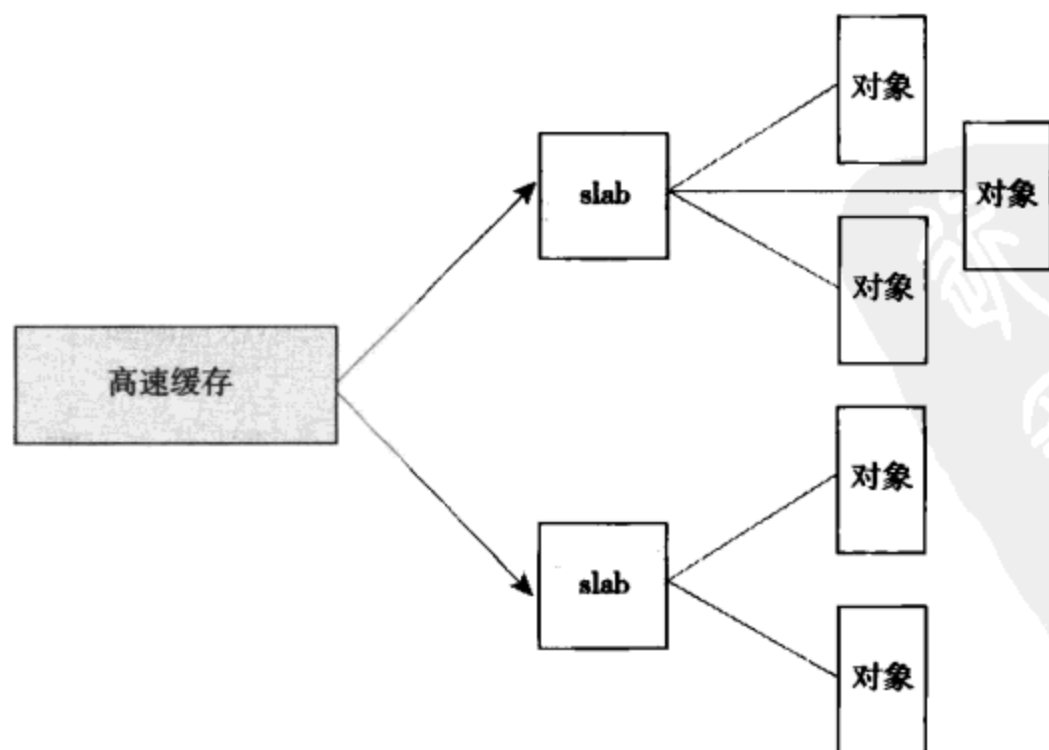


图 12-1 高速缓存、slab 及对象之间的关系

每个高速缓存都使用 `kmem_cache` 结构来表示。这个结构包含三个链表：`slabs_full`、`slabs_partial` 和 `slabs_empty`，均存放在 `kmem_list3` 结构内，该结构在 `mm/slab.c` 中定义。这些链表包含高速缓存中的所有 slab。slab 描述符 `struct slab` 用来描述每个 slab：

```
struct slab {
    struct list_head list;           /* 满、部分满或空链表 */
    unsigned long colouroff;        /* slab 着色的偏移量 */
    void *s_mem;                    /* 在 slab 中的第一个对象 */
    unsigned int inuse;              /* slab 中已分配的对象数 */
    kmem_bufctl_t free;             /* 第一个空闲对象（如果有的话） */
};
```

slab 描述符要么在 slab 之外另行分配，要么就放在 slab 自身开始的地方。如果 slab 很小，或者 slab 内部有足够的空间容纳 slab 描述符，那么描述符就存放在 slab 里面。

slab 分配器可以创建新的 slab，这是通过 `__get_free_pages()` 低级内核页分配器进行的：

```
static void *kmem_getpages(struct kmem_cache *cachep, gfp_t flags, int nodeid)
{
    struct page *page;
    void *addr;
    int i;

    flags |= cachep->gfpflags;
    if (likely(nodeid == -1)) {
        addr = (void *)__get_free_pages(flags, cachep->gfporder);
        if (!addr)
            return NULL;
        page = virt_to_page(addr);
    } else {
        page = alloc_pages_node(nodeid, flags, cachep->gfporder);
        if (!page)
            return NULL;
        addr = page_address(page);
    }

    i = (1 << cachep->gfporder);
    if (cachep->flags & SLAB_RECLAIM_ACCOUNT)
        atomic_add(i, &slab_reclaim_pages);
    add_page_state(nr_slab, i);
    while (i-- > 0) {
        SetPageSlab(page);
        page++;
    }
    return addr;
}
```

该函数使用 `__get_free_pages()` 来为高速缓存分配足够多的内存。该函数的第一个参数就指向需要很多页的特定高速缓存。第二个参数是要传给 `__get_free_pages()` 的标志，注意这个标志是如何与另一个值进行二进制“或”运算的，这相当于把高速缓存需要的缺省标志加到 `flags` 参数上。

分配的页大小为2的幂次方，存放在 `cachep->gfporder` 中。由于与分配器 NUMA 相关的代码的关系前面这个函数比想象的要复杂一些。当 `nodeid` 是一个非负数时，分配器就试图对从相同的内存节点给发出的请求进行分配。这在 NUMA 系统上提供了较好的性能，但是访问节点之外的内存会导致性能的损失。

为了便于理解，我们可以忽略与 NUMA 相关的代码，写一个简单的 `kmem_getpages()` 函数：

```
static inline void * kmem_getpages(struct kmem_cache *cachep, gfp_t flags)
{
    void *addr;

    flags |= cachep->gfpflags;
    addr = (void*) __get_free_pages(flags, cachep->gfporder);

    return addr;
}
```

接着，调用 `kmem_freepages()` 释放内存，而对给定的高速缓存页，`kmem_freepages()` 最终调用的是 `free_pages()`。当然，slab 层的关键就是避免频繁分配和释放页。由此可知，slab 层只有当给定的高速缓存部分中既没有满也没有空的 slab 时才会调用页分配函数。而只有在下列情况下才会调用释放函数：当可用内存变得紧缺时，系统试图释放出更多内存以供使用；或者当高速缓存显式地被撤销时。

slab 层的管理是在每个高速缓存的基础上，通过提供给整个内核一个简单的接口来完成的。通过接口就可以创建和撤销新的高速缓存，并在高速缓存内分配和释放对象。高速缓存及其内 slab 的复杂管理完全通过 slab 层的内部机制来处理。当你创建了一个高速缓存后，slab 层所起的作用就像一个专用的分配器，可以为具体的对象类型进行分配。

12.6.2 slab 分配器的接口

一个新的高速缓存通过以下函数创建：

```
struct kmem_cache * kmem_cache_create(const char *name,
                                     size_t size,
                                     size_t align,
                                     unsigned long flags,
                                     void (*ctor)(void *));
```

第一个参数是一个字符串，存放着高速缓存的名字；第二个参数是高速缓存中每个元素的大小；第三个参数是 slab 内第一个对象的偏移，它用来确保在页内进行特定的对齐。通常情况下，0 就可以满足要求，也就是标准对齐。flags 参数是可选的设置项，用来控制高速缓存的行为。它可以为 0，表示没有特殊的行为，或者与以下标志中的一个或多个进行“或”运算：

- **SLAB_HWCACHE_ALIGN**——这个标志命令 slab 层把一个 slab 内的所有对象按高速缓存行对齐。这就防止了“错误的共享”（两个或多个对象尽管位于不同的内存地址，但映射到相同的高速缓存行）。这可以提高性能，但以增加内存开销为代价，因为对齐越严格，

浪费的内存就越多。到底会耗费掉多少内存，取决于对象的大小以及对象相对于系统高速缓存行对齐的方式。对于会频繁使用的高速缓存，而且代码本身对性能要求又很严格的情况，设置该选项是理想的选择；否则，请三思而后行。

- **SLAB_POISON**——这个标志使 slab 层用已知的值 (a5a5a5a5) 填充 slab。这就是所谓的“中毒”，有利于对未初始化内存的访问。
- **SLAB_RED_ZONE**——这个标志导致 slab 层在已分配的内存周围插入“红色警戒区”以探测缓冲越界。
- **SLAB_PANIC**——这个标志当分配失败时提醒 slab 层。这在要求分配只能成功的时候非常有用。比如，在系统初启时分配一个 VMA 结构的高速缓存（参见第 15 章）。
- **SLAB_CACHE_DMA**——这个标志命令 slab 层使用可以执行 DMA 的内存给每个 slab 分配空间。只有在分配的对象用于 DMA，而且必须驻留在 ZONE_DMA 区时才需要这个标志。否则，你既不需要也不应该设置这个标志。

最后一个参数 `ctor` 是高速缓存的构造函数。只有在新的页追加到高速缓存时，构造函数才被调用。实际上，Linux 内核的高速缓存不使用构造函数。事实上这里曾经还有一个析构函数参数，但是由于内核代码并不需要它，因此已经被抛弃了。你可以将 `ctor` 参数赋值为 `NULL`。

`kmem_cache_create()` 在成功时会返回一个指向所创建高速缓存的指针；否则，返回 `NULL`。这个函数不能在中断上下文中调用，因为它可能会睡眠。

要撤销一个高速缓存，则调用：

```
int kmem_cache_destroy(struct kmem_cache *cachep)
```

顾名思义，这样就可以撤销给定的高速缓存。这个函数通常在模块的注销代码中被调用，当然，这里指创建了自己的高速缓存的模块。同样，也不能从中断上下文中调用这个函数，因为它也可能睡眠。调用该函数之前必须确保存在以下两个条件：

- 高速缓存中的所有 slab 都必须为空。其实，不管哪个 slab 中，只要还有一个对象被分配出去并正在使用的话，那怎么可能撤销这个高速缓存呢？
- 在调用 `kmem_cache_destroy()` 过程中（更不用说在调用之后了）不再访问这个高速缓存。调用者必须确保这种同步。

该函数在成功时返回 0，否则返回非 0 值。

1. 从缓存中分配

创建高速缓存之后，就可以通过下列函数获取对象：

```
void * kmem_cache_alloc(struct kmem_cache *cachep, gfp_t flags)
```

该函数从给定的高速缓存 `cachep` 中返回一个指向对象的指针。如果高速缓存的所有 slab 中都没有空闲的对象，那么 slab 层必须通过 `kmem_getpages()` 获取新的页，`flags` 的值传递给 `_get_free_pages()`。这与我们前面看到的标志相同，你用到的应该是 `GFP_KERNEL` 或 `GFP_ATOMIC`。

最后释放一个对象，并把它返回给原先的 slab，可以使用下面这个函数：

```
void kmem_cache_free(struct kmem_cache *cachep, void *objp)
```

这样就能把 cachep 中的对象 objp 标记为空闲。

2. slab 分配器的使用实例

让我们考察一个鲜活的实例，这个例子用的是 task_struct 结构（进程描述符）。代码稍微有点复杂，取自 kernel/fork.c。

首先，内核用一个全局变量存放指向 task_struct 高速缓存的指针：

```
struct kmem_cache *task_struct_cachep;
```

在内核初始化期间，在定义于 kernel/fork.c 的 fork_init() 中会创建高速缓存：

```
task_struct_cachep = kmem_cache_create("task_struct",
                                       sizeof(struct task_struct),
                                       ARCH_MIN_TASKALIGN,
                                       SLAB_PANIC | SLAB_NOTRACK,
                                       NULL);
```

这样就创建了一个名为 task_struct 的高速缓存，其中存放的就是类型为 struct task_struct 的对象。该对象被创建后存放在 slab 中偏移量为 ARCH_MIN_TASKALIGN 个字节的地方，ARCH_MIN_TASKALIGN 预定义值与体系结构相关。通常将它定义为 L1_CACHE_BYTES——L1 高速缓存的字节大小。没有构造函数或析构函数。注意不用检查返回值是否为失败标记 NULL，因为 SLAB_PANIC 标志已经被设置了。如果分配失败，slab 分配器就调用 panic() 函数。如果没有提供 SLAB_PANIC 标志，就必须自己检查返回值。SLAB_PANIC 标志用在这儿是因为这是系统操作必不可少的高速缓存（没有进程描述符，机器自然不能正常运行）。

每当进程调用 fork() 时，一定会创建一个新的进程描述符（回忆一下第 3 章）。这是在 dup_task_struct() 中完成的，而该函数会被 do_fork() 调用：

```
struct task_struct *tsk;

tsk = kmem_cache_alloc(task_struct_cachep, GFP_KERNEL);
if (!tsk)
    return NULL;
```

进程执行完后，如果没有子进程在等待的话，它的进程描述符就会被释放，并返回给 task_struct_cachep slab 高速缓存。这是在 free_task_struct() 中执行的（这里，tsk 是现有的进程）：

```
kmem_cache_free(task_struct_cachep, tsk);
```

由于进程描述符是内核的核心组成部分，时刻都要用到，因此 task_struct_cachep 高速缓存绝不会被撤销掉。即使真能撤销，我们也要通过下列函数阻止其被撤销：

```
int err;

err = kmem_cache_destroy(task_struct_cachep);
if (err)
```

```
/* 出错，撤销高速缓存 */
```

很容易吧？slab 层负责内存紧缺情况下所有底层的对齐、着色、分配、释放和回收等。如果你要频繁创建很多相同类型的对象，那么，就应该考虑使用 slab 高速缓存。也就是说，不要自己去实现空闲链表！

12.7 在栈上的静态分配

在用户空间，我们以前所讨论到的那些分配的例子，有不少都可以在栈上发生。因为我们毕竟可以事先知道所分配空间的大小。用户空间能够奢侈地负担起非常大的栈，而且栈空间还可以动态增长，相反，内核却不能这么奢侈——内核栈小而且固定。当给每个进程分配一个固定大小的小栈后，不但可以减少内存的消耗，而且内核也无须负担太重的栈管理任务。

每个进程的内核栈大小既依赖体系结构，也与编译时的选项有关。历史上，每个进程都有两页的内核栈。因为 32 位和 64 位体系结构的页面大小分别是 4KB 和 8KB，所以通常它们的内核栈的大小分别是 8KB 和 16KB。

12.7.1 单页内核栈

但是，在 2.6 系列内核的早期，引入了一个选项设置单页内核栈。当激活这个选项时，每个进程的内核栈只有一页那么大，根据体系结构的不同，或为 4KB，或为 8KB。这么做出于两个原因：首先，可以让每个进程减少内存消耗。其次，也是最重要的，随着机器运行时间的增加，寻找两个未分配的、连续的页变得越来越困难。物理内存渐渐变为碎片，因此，给一个新进程分配虚拟内存（VM）的压力也在增大。

还有一个更复杂的原因。继续跟随我：我们几乎掌握了关于内核栈的全部知识。现在，每个进程的整个调用链必须放在自己的内核栈中。不过，中断处理程序也曾经使用它们所中断的进程的内核栈，这样，中断处理程序也要放在内核栈中。这当然有效而简单，但是，这同时会把更严格的约束条件加在这可怜的内核栈上。当我们转而使用只有一个页面的内核栈时，中断处理程序就不放在栈中了。

为了矫正这个问题，内核开发者们实现了一个新功能：中断栈。中断栈为每个进程提供一个用于中断处理程序的栈。有了这个选项，中断处理程序不用再和被中断进程共享一个内核栈，它们可以使用自己的栈了。对每个进程来说仅仅耗费了一页而已。

总的来说，内核栈可以是 1 页，也可以是 2 页，这取决于编译时配置选项。栈大小因此在 4 ~ 16KB 的范围内。历史上，中断处理程序和被中断进程共享一个栈。当 1 页栈的选项激活时，中断处理程序获得了自己的栈。在任何情况下，无限制的递归和 `alloca()` 显然是不被允许的。

好，就讲到这里。大家明白了吗？

12.7.2 在栈上光明正大地工作

在任意一个函数中，你都必须尽量节省栈资源。这并不难，也没有什么窍门，只需要在具体的函数中让所有局部变量（即所谓的自动变量）所占空间之和不要超过几百字节。在栈上进行大

量的静态分配（比如分配大型数组或大型结构体）是很危险的。要不然，在内核中和在用户空间中进行的栈分配就没有什么差别了。栈溢出时悄无声息，但势必会引起严重的问题。因为内核没有管理内核栈上做足工作，因此，当栈溢出时，多出的数据就会直接溢出来，覆盖掉紧邻堆栈末端的东西。首先面临考验的就是 `thread_info` 结构（回想一下第 3 章，这个结构就贴着每个进程内核堆栈的末端）。在堆栈之外，任何内核数据都可能存在潜在的危险。当栈溢出时，最好的情况是机器宕机，最坏的情况是悄无声息地破坏数据。

因此，进行动态分配是一种明智的选择，本章前面有关大块内存的分配就是采用这种方式。

12.8 高端内存的映射

根据定义，在高端内存中的页不能永久地映射到内核地址空间上。因此，通过 `alloc_pages()` 函数以 `__GFP_HIGHMEM` 标志获得的页不可能有逻辑地址。

在 x86 体系结构上，高于 896MB 的所有物理内存的范围大都是高端内存，它并不会永久地或自动地映射到内核地址空间，尽管 x86 处理器能够寻址物理 RAM 的范围达到 4GB（启用 PAE[⊖]可以寻址到 64GB）。一旦这些页被分配，就必须映射到内核的逻辑地址空间上。在 x86 上，高端内存中的页被映射到 3GB ~ 4GB。

12.8.1 永久映射

要映射一个给定的 `page` 结构到内核地址空间，可以使用定义在文件 `<linux/highmem.h>` 中的这个函数：

```
void *kmap(struct page *page)
```

这个函数在高端内存或低端内存上都能用。如果 `page` 结构对应的是低端内存中的一页，函数只会单纯地返回该页的虚拟地址。如果页位于高端内存，则会建立一个永久映射，再返回地址。这个函数可以睡眠，因此 `kmap()` 只能用在进程上下文中。

因为允许永久映射的数量是有限的（如果没有这个限制，我们就不必搞得这么复杂，把所有内存通通映射为永久内存就行了），当不再需要高端内存时，应该解除映射，这可以通过下列函数完成：

```
void kunmap(struct page *page)
```

12.8.2 临时映射

当必须创建一个映射而当前的上下文又不能睡眠时，内核提供了临时映射（也就是所谓的原子映射）。有一组保留的映射，它们可以存放新创建的临时映射。内核可以原子地把高端内存中的一个页映射到某个保留的映射中。因此，临时映射可以用在不能睡眠的地方，比如中断处理程序中，因为获取映射时绝不会阻塞。

⊖ PAE 是 Physical Address Extension 的缩写，这是 x86 处理器的特点，这种特点使得 x86 处理器尽管只有 32 位的虚拟地址空间，但从物理上能寻址到 36 位（64GB）的内存空间。

通过下列函数建立一个临时映射：

```
void *kmap_atomic(struct page *page, enum km_type type)
```

参数 `type` 是下列枚举类型之一，这些枚举类型描述了临时映射的目的。它们定义于 `<asm/kmap_types.h>` 中：

```
enum km_type {
    KM_BOUNCE_READ,
    KM_SKB_SUNRPC_DATA,
    KM_SKB_DATA_SOFTIRQ,
    KM_USER0,
    KM_USER1,
    KM_BIO_SRC_IRQ,
    KM_BIO_DST_IRQ,
    KM_PTE0,
    KM_PTE1,
    KM_PTE2,
    KM_IRQ0,
    KM_IRQ1,
    KM_SOFTIRQ0,
    KM_SOFTIRQ1,
    KM_SYNC_ICACHE,
    KM_SYNC_DCACHE,
    KM_UML_USERCOPY,
    KM_IRQ_PTE,
    KM_NMI,
    KM_NMI_PTE,
    KM_TYPE_NR
};
```

这个函数不会阻塞，因此可以用在中断上下文和其他不能重新调度的地方。它也禁止内核抢占，这是有必要的，因为映射对每个处理器都是唯一的（调度可能对哪个处理器执行哪个进程做变动）。

通过下列函数取消映射：

```
void kunmap_atomic(void *kvaddr, enum km_type type)
```

这个函数也不会阻塞。在很多体系结构中，除非激活了内核抢占，否则 `kmap_atomic()` 根本就无事可做，因为只有在下一个临时映射到来前上一个临时映射才有效。因此，内核完全可以“忘掉” `kmap_atomic()` 映射，`kunmap_atomic()` 也无须做什么实际的事情。下一个原子映射将自动覆盖前一个映射。

12.9 每个 CPU 的分配

支持 SMP 的现代操作系统使用每个 CPU 上的数据，对于给定的处理器其数据是唯一的。一般来说，每个 CPU 的数据存放在一个数组中。数组中的每一项对应着系统上一个存在的处理器。按当前处理器号确定这个数组的当前元素，这就是 2.4 内核处理每个 CPU 数据的方式。这种方

式还不错，因此，2.6 内核的很多代码依然用它。可以声明数据如下：

```
unsigned long my_percpu[NR_CPUS];
```

然后，按如下方式访问它：

```
int cpu;
```

```
cpu = get_cpu();           /* 获得当前处理器，并禁止内核抢占 */
my_percpu[cpu]++;          /* ... 或者无论什么 */
printk("my_percpu on cpu=%d is %lu\n", cpu, my_percpu[cpu]);
put_cpu();                 /* 激活内核抢占 */
```

注意，上面的代码中并没有出现锁，这是因为所操作的数据对当前处理器来说是唯一的。除了当前处理器之外，没有其他处理器可接触到这个数据，不存在并发访问问题，所以当前处理器可以在不用锁的情况下安全访问它。

现在，内核抢占成为了唯一需要关注的问题了，内核抢占会引起下面提到的两个问题：

- 如果你的代码被其他处理器抢占并重新调度，那么这时 CPU 变量就会无效，因为它指向的是错误的处理器（通常，代码获得当前处理器后是不可以睡眠的）。
- 如果另一个任务抢占了你的代码，那么有可能在同一个处理器上发生并发访问 my_percpu 的情况，显然这属于一个竞争条件。

虽然如此，但是你大可不必惊慌，因为在获取当前处理器号，即调用 get_cpu() 时，就已经禁止了内核抢占。相应的在调用 put_cpu() 时又会重新激活当前处理器号。注意，只要你总使用上述方法来保护数据安全，那么，内核抢占就不需要你自己去禁止。

12.10 新的每个 CPU 接口

2.6 内核为了方便创建和操作每个 CPU 数据，而引进了新的操作接口，称作 percpu。该接口归纳了前面所述的操作行为，简化了创建和操作每个 CPU 的数据。

但前面我们讨论的创建和访问每个 CPU 的方法依然有效，不过大型对称多处理器计算机要求对每个 CPU 数据操作更简单，功能更强大，正是在这种背景下，新接口应运而生。

头文件 <linux/percpu.h> 声明了所有的接口操作例程，你可以在文件 mm/slab.c 和 <asm/percpu.h> 中找到它们的定义。

12.10.1 编译时的每个 CPU 数据

在编译时定义每个 CPU 变量易如反掌：

```
DEFINE_PER_CPU(type, name);
```

这个语句为系统中的每一个处理器都创建了一个类型为 type，名字为 name 的变量实例，如果你需要在别处声明变量，以防范编译时警告，那么下面的宏将是你的好帮手：

```
DECLARE_PER_CPU(type, name);
```

你可以利用 get_cpu_var() 和 put_cpu_var() 例程操作变量。调用 get_cpu_var() 返回当前处理

器上的指定变量，同时它将禁止抢占；另一方面 `put_cpu_var()` 将相应的重新激活抢占。

```
get_cpu_var(name)++;      /* 增加该处理器上的 name 变量的值 */
put_cpu_var(name);        /* 完成；重新激活内核抢占 */
```

你也可以获得别的处理器上的每个 CPU 数据：

```
per_cpu(name, cpu)++;     /* 增加指定处理器上的 name 变量的值 */
```

使用此方法你需要格外小心，因为 `per_cpu()` 函数既不会禁止内核抢占，也不会提供任何形式的锁保护。如果一些处理器可以接触到其他处理器的数据，那么你就必须要给数据上锁。注意，第 9 章和第 10 章详细讨论了数据上锁问题。

另外还有一个需要提醒的问题：这些编译时每个 CPU 数据的例子并不能在模块内使用，因为连接程序实际上将它们创建在一个唯一的可执行段中（`.data.percpu`）。如果你需要从模块中访问每个 CPU 数据，或者如果你需要动态创建这些数据，那还是有希望的。

12.10.2 运行时的每个 CPU 数据

内核实现每个 CPU 数据的动态分配方法类似于 `kmalloc()`。该例程为系统上的每个处理器创建所需内存的实例，其原型在文件 `<linux/percpu.h>` 中：

```
void *alloc_percpu(type); /* 一个宏 */
void *__alloc_percpu(size_t size, size_t align);
void free_percpu(const void *);
```

宏 `alloc_percpu()` 给系统中的每个处理器分配一个指定类型对象的实例。它其实是宏 `__alloc_percpu()` 的一个封装，这个原始宏接收的参数有两个：一个是要分配的实际字节数，一个是分配时要按多少字节对齐。而封装后的 `alloc_percpu()` 按照单字节对齐——按照给定类型的自然边界对齐。这种对齐方式最为常用。比如：

```
struct rabid_cheetah = alloc_percpu(struct rabid_cheetah);
```

它等价于

```
struct rabid_cheetah = __alloc_percpu(sizeof (struct rabid_cheetah),
__alignof__ (struct rabid_cheetah));
```

`__alignof__` 是 gcc 的一个功能，它会返回指定类型或 lvalue 所需的（或建议的，要知道有些古怪的体系结构并没有字节对齐的要求）对齐字节数。它的语义和 `sizeof` 一样，比如，下列程序在 x86 体系中将返回 4：

```
__alignof__ (unsigned long)
```

如果指定一个 lvalue，那么将返回 lvalue 的最大对齐字节数。比如一个结构中的 lvalue 相比结构外的 lvalue 可能有更大的对齐字节需求，这是结构本身的对齐要求的缘故。有关对齐的进一步讨论我们放在第 19 章中介绍。

相应的调用 `free_percpu()` 将释放所有处理器上指定的每个 CPU 数据。

无论是 `alloc_percpu()` 或是 `__alloc_percpu()` 都会返回一个指针，它用来间接引用动态创建的每个 CPU 数据，内核提供了两个宏来利用指针获取每个 CPU 数据：

```
get_cpu_var(ptr);          /* 返回一个 void 类型指针，该指针指向处理器的 ptr 的拷贝 */
put_cpu_var(ptr);          /* 完成：重新激活内核抢占 */
```

`get_cpu_var()` 宏返回了一个指向当前处理器数据的特殊实例，它同时会禁止内核抢占；而在 `et_cpu_var()` 宏中会重新激活内核抢占。

我们来看一个使用这些函数的完整例子。当然这个例子有点无聊，因为通常你会一次分配够内存（比如，在某些初始化函数中），就可以在各种地方使用它，或再一次释放（比如，在一些清理函数中）。不过，这个例子可清楚地说明如何使用这些函数。

```
void *percpu_ptr;
unsigned long *foo;

percpu_ptr = alloc_percpu(unsigned long);
if (!ptr)
    /* 内存分配错误 ... */

foo = get_cpu_var(percpu_ptr);
/* 操作 foo ... */
put_cpu_var(percpu_ptr);
```

12.11 使用每个 CPU 数据的原因

使用每个 CPU 数据具有不少好处。首先是减少了数据锁定。因为按照每个处理器访问每个 CPU 数据的逻辑，你可以不再需要任何锁。记住“只有这个处理器能访问这个数据”的规则纯粹是一个编程约定。你需要确保本地处理器只会访问它自己的唯一数据。系统本身并不存在任何措施禁止你从事欺骗活动。

第二个好处是使用每个 CPU 数据可以大大减少缓存失效。失效发生在处理器试图使它们的缓存保持同步时。如果一个处理器操作某个数据，而该数据又存放在其他处理器缓存中，那么存放该数据的那个处理器必须清理或刷新自己的缓存。持续不断的缓存失效称为缓存抖动，这样对系统性能影响颇大。使用每个 CPU 数据将使得缓存影响降至最低，因为理想情况下只会访问自己的数据。`percpu` 接口缓存一对齐（`cache-align`）所有数据，以便确保在访问一个处理器的数据时，不会将另一个处理器的数据带入同一个缓存线上。

综上所述，使用每个 CPU 数据会省去许多（或最小化）数据上锁，它唯一的安全要求就是要禁止内核抢占。而这点代价相比上锁要小得多，而且接口会自动帮你完成这个步骤。每个 CPU 数据在中断上下文或进程上下文中使用都很安全。但要注意，不能在访问每个 CPU 数据过程中睡眠——否则，你就可能醒来后已经到了其他处理器上了。

目前并不要求必须使用每个 CPU 的新接口。只要你禁止了内核抢占，用手动方法（利用我们原来讨论的数组）就很好，但是新接口在将来更容易使用，而且功能也会得到长足的优化。如果确实决定在你的内核中使用每个 CPU 数据，请考虑使用新接口。但我要提醒的是——新接口并不向后兼容之前的内核。

12.12 分配函数的选择

在这么多分配函数和方法中，有时并不能搞清楚到底该选择那种方式分配——但这确实很重要。如果你需要连续的物理页，就可以使用某个低级页分配器或 `kmalloc()`。这是内核中内存分配的常用方式，也是大多数情况下你自己应该使用的内存分配方式。回忆一下，传递给这些函数的两个最常用的标志是 `GFP_ATOMIC` 和 `GFP_KERNEL`。`GFP_ATOMIC` 表示进行不睡眠的高优先级分配，这是中断处理程序和其他不能睡眠的代码段的需要。对于可以睡眠的代码，（比如没有持自旋锁的进程上下文代码）则应该使用 `GFP_KERNEL` 获取所需的内存。这个标志表示如果有必要，分配时可以睡眠。

如果你想从高端内存进行分配，就使用 `alloc_pages()`。`alloc_pages()` 函数返回一个指向 `struct page` 结构的指针，而不是一个指向某个逻辑地址的指针。因为高端内存很可能并没有被映射，因此，访问它的唯一方式就是通过相应的 `struct page` 结构。为了获得真正的指针，应该调用 `kmap()`，把高端内存映射到内核的逻辑地址空间。

如果你不需要物理上连续的页，而仅仅需要虚拟地址上连续的页，那么就使用 `vmalloc()`（不过要记住 `vmalloc()` 相对 `kmalloc()` 来说，有一定的性能损失）。`vmalloc()` 函数分配的内存虚地址是连续的，但它本身并不保证物理上的连续。这与用户空间的分配非常类似，它也是把物理内存块映射到连续的逻辑地址空间上。

如果你要创建和撤销很多大的数据结构，那么考虑建立 slab 高速缓存。slab 层会给每个处理器维持一个对象高速缓存（空闲链表），这种高速缓存会极大地提高对象分配和回收的性能。slab 层不是频繁地分配和释放内存，而是为你把事先分配好的对象存放到高速缓存中。当你需要一块新的内存来存放数据结构时，slab 层一般无须另外去分配内存，而只需要从高速缓存中得到一个对象就可以了。

12.13 小结

本章中，我们学习了 Linux 内核如何管理内存。我们首先看到了内存空间的各种不同的描述单位，包括字节、页面和区（在第 15 章的进程地址空间中可看到 4 种不同层次的内存单位）。我们接着讨论了各种内存分配机制，其中包括页分配器和 slab 分配器。在内核中分配内存并非总是轻而易举，因为你必须小心地确保分配过程遵从内核特定的状态约束。比如分配过程中不得堵塞，或者访问文件系统等约束。为此我们讨论了 gfp 标识以及使用每个标识的针对场景。分配内存相对复杂是内核开发和用户程序开发的最大区别之一，本章使用大量篇幅描述内存分配的各种不同接口——通过这些不同调用接口，你应该能感觉到内核中分配内存为什么更复杂的原因。在本章基础上，在第 13 章我们讨论虚拟文件系统（VFS）——负责管理文件系统且为用户空间程序提供一致性接口的内核子系统。我们继续深入！

第 13 章

虚拟文件系统

虚拟文件系统（有时也称作虚拟文件交换，更常见的是简称 VFS）作为内核子系统，为用户空间程序提供了文件和文件系统相关的接口。系统中所有文件系统不但依赖 VFS 共存，而且也依靠 VFS 系统协同工作。通过虚拟文件系统，程序可以利用标准的 Unix 系统调用对不同的文件系统，甚至不同介质上的文件系统进行读写操作，如图 13-1 所示。

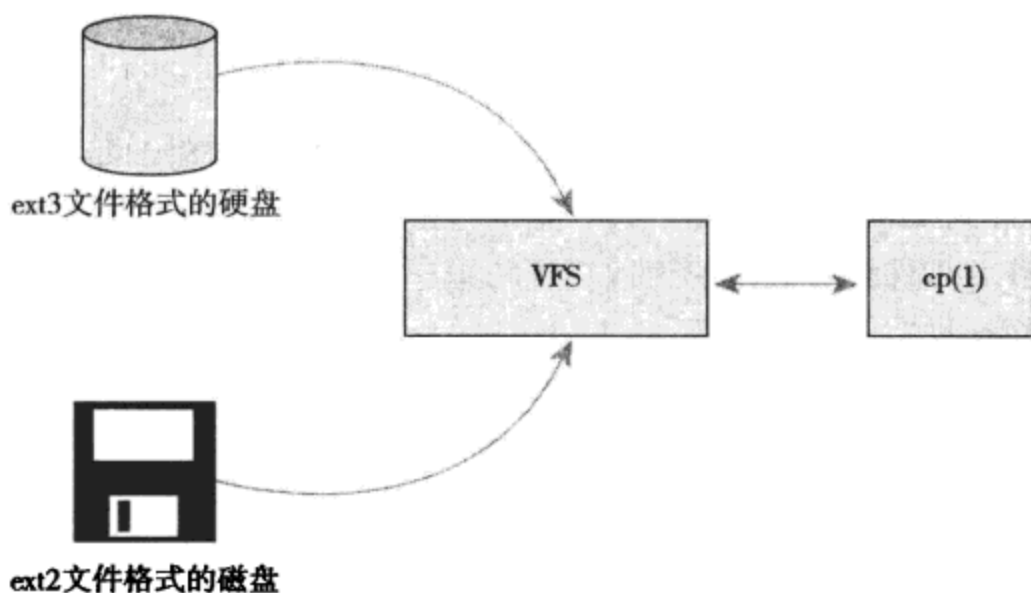


图 13-1 VFS 执行的动作：使用 `cp(1)` 命令从 ext3 文件系统格式的硬盘拷贝数据到 ext2 文件系统格式的可移动磁盘上。两种不同的文件系统，两种不同的介质，连接到同一个 VFS 上

13.1 通用文件系统接口

VFS 使得用户可以直接使用 `open()`、`read()` 和 `write()` 这样的系统调用而无须考虑具体文件系统和实际物理介质。现在听起来这也没什么新奇的（我们早就认为这是理所当然的），但是，使得这些通用的系统调用可以跨越各种文件系统和不同介质执行，绝非是微不足道的成绩。更了不起的是，系统调用可以在这些不同的文件系统和介质之间执行——我们可以使用标准的系统调用从一个文件系统拷贝或移动数据到另一个文件系统。老式的操作系统（比如 DOS）是无力完成上述工作的，任何对非本地文件系统的访问都必须依靠特殊工具才能完成。正是由于现代操作系统引入抽象层，比如 Linux，通过虚拟接口访问文件系统，才使得这种协作性和泛型存取成为可能。

新的文件系统和新类型的存储介质都能找到进入 Linux 之路，程序无需重写，甚至无须重新编译。在本章中，我们将讨论 VFS，它把各种不同的文件系统抽象后采用统一的方式进行操作。在第 14 章中，我们将讨论块 I/O 层，它支持各种各样的存储设备——从 CD 到蓝光光盘，从硬件设备再到压缩闪存。VFS 与块 I/O 相结合，提供抽象、接口以及交融，使得用户空间的程序调

用统一的系统调用访问各种文件，不管文件系统是什么，也不管文件系统位于何种介质，采用的命名策略是统一的。

13.2 文件系统抽象层

之所以可以使用这种通用接口对所有类型的文件系统进行操作，是因为内核在它的底层文件系统接口上建立了一个抽象层。该抽象层使 Linux 能够支持各种文件系统，即便是它们在功能和行为上存在很大差别。为了支持多文件系统，VFS 提供了一个通用文件系统模型，该模型囊括了任何文件系统的常用功能集和行为。当然，该模型偏重于 Unix 风格的文件系统（我们将在后面的小节看到 Unix 风格的文件系统的构成）。但即使这样，Linux 仍然可以支持很多种差异很大的文件系统，从 DOS 系统的 FAT 到 Windows 系统的 NTFS，再到各种 Unix 风格文件系统和 Linux 特有的文件系统。

VFS 抽象层之所以能衔接各种各样的文件系统，是因为它定义了所有文件系统都支持的、基本的、概念上的接口和数据结构。同时实际文件系统也将自身的诸如“如何打开文件”，“目录是什么”等概念在形式上与 VFS 的定义保持一致。因为实际文件系统的代码在统一的接口和数据结构下隐藏了具体的实现细节，所以在 VFS 层和内核的其他部分看来，所有文件系统都是相同的，它们都支持像文件和目录这样的概念，同时也支持像创建文件和删除文件这样的操作。

内核通过抽象层能够方便、简单地支持各种类型的文件系统。实际文件系统通过编程提供 VFS 所期望的抽象接口和数据结构，这样，内核就可以毫不费力地和任何文件系统协同工作，并且这样提供给用户空间的接口，也可以和任何文件系统无缝地连接在一起，完成实际工作。

其实在内核中，除了文件系统本身外，其他部分并不需要了解文件系统的内部细节。比如一个简单的用户空间程序执行如下的操作：

```
ret = write(fd, buf, len);
```

该系统调用将 buf 指针指向的长度为 len 字节的数据写入文件描述符 fd 对应的文件的当前位置。这个系统调用首先被一个通用系统调用 sys_write() 处理，sys_write() 函数要找到 fd 所在的文件系统实际给出的是哪个写操作，然后再执行该操作。实际文件系统的写方法是文件系统实现的一部分，数据最终通过该操作写入介质（或执行这个文件系统想要完成的写动作）。图 13-2 描述了从用户空间的 write() 调用到数据被写入磁盘介质的整个流程。一方面，系统调用是通用 VFS 接口，提供给用户空间的前端；另一方面，系统调用是具体文件系统的后端，处理实现细节。接下来的小节中我们会具体看到 VFS 抽象模型以及它提供的接口。

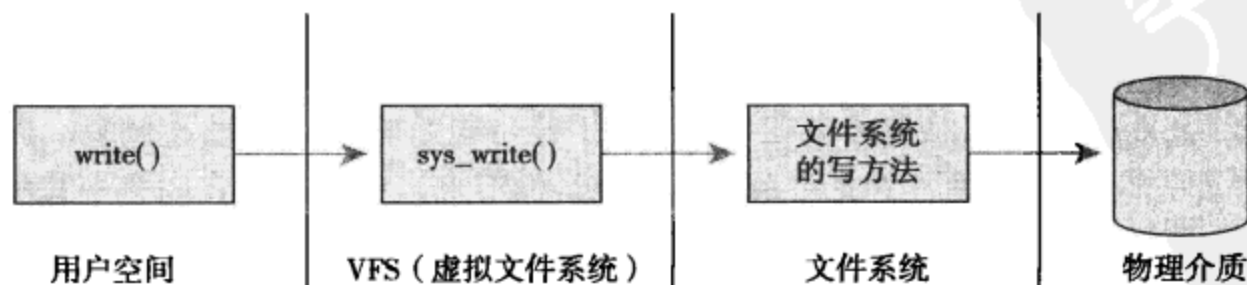


图 13-2 write() 调用将来自用户空间的数据流，首先通过 VFS 的通用系统调用，其次通过文件系统的特殊写法，最后写入物理介质中

13.3 Unix 文件系统

Unix 使用了四种和文件系统相关的传统抽象概念：文件、目录项、索引节点和安装点 (mount point)。

从本质上讲文件系统是特殊的数据分层存储结构，它包含文件、目录和相关的控制信息。文件系统的通用操作包含创建、删除和安装等。在 Unix 中，文件系统被安装在一个特定的安装点上，该安装点在全局层次结构^①中被称作命名空间，所有的已安装文件系统都作为根文件系统树的枝叶出现在系统中。与这种单一、统一的树形成鲜明对照的就是 DOS 和 Windows 的表现，它们将文件的命名空间分类为驱动字母，例如 C:。这种将命名空间划分为设备和分区的做法，相当于把硬件细节“泄露”给文件系统抽象层。对用户而言，如此的描述有点随意，甚至产生混淆，这是 Linux 统一命名空间所不屑一顾的。

文件其实可以做一个有序字节串，字节串中第一个字节是文件的头，最后一个字节是文件的尾。每一个文件为了便于系统和用户识别，都被分配了一个便于理解的名字。典型的文件操作有读、写、创建和删除等。Unix 文件的概念与面向记录的文件系统（如 OpenVMS 的 File-11）形成鲜明的对照。面向记录的文件系统提供更丰富、更结构化的表示，而简单的面向字节流抽象的 Unix 文件则以简单性和相当的灵活性为代价。

文件通过目录组织起来。文件目录好比一个文件夹，用来容纳相关文件。因为目录也可以包含其他目录，即子目录，所以目录可以层层嵌套，形成文件路径。路径中的每一部分都被称作目录条目。“/home/wolfman/butter”是文件路径的一个例子——根目录 /，目录 home，wolfman 和文件 butter 都是目录条目，它们统称为目录项。在 Unix 中，目录属于普通文件，它列出包含在其中的所有文件。由于 VFS 把目录当作文件对待，所以可以对目录执行和文件相同的操作。

Unix 系统将文件的相关信息和文件本身这两个概念加以区分，例如访问控制权限、大小、拥有者、创建时间等信息。文件相关信息，有时被称作文件的元数据（也就是说，文件的相关数据），被存储在一个单独的数据结构中，该结构被称为索引节点 (inode)，它其实是 index node 的缩写，不过近来术语“inode”使用得更为普遍一些。

所有这些信息都和文件系统的控制信息密切相关，文件系统的控制信息存储在超级块中，超级块是一种包含文件系统信息的数据结构。有时，把这些收集起来的信息称为文件系统数据元，它集单独文件信息和文件系统的信息于一身。

一直以来，Unix 文件系统在它们物理磁盘布局中也是按照上述概念实现的。比如说在磁盘上，文件（目录也属于文件）信息按照索引节点形式存储在单独的块中；控制信息被集中存储在磁盘的超级块中，等等。Unix 中文件的概念从物理上被映射到存储介质。Linux 的 VFS 的设计目标就是要保证能与支持和实现了这些概念的文件系统协同工作。像如 FAT 或 NTFS 这样的非 Unix 风格的文件系统，虽然也可以在 Linux 上工作，但是它们必须经过封装，提供一个符合这些概念的界面。比如，即使一个文件系统不支持索引节点，它也必须在内存中装配索引节点结构体，就像它本身包含索引节点一样。再比如，如果一个文件系统将目录看做一种特殊对象，那

^① 近来，Linux 已经将这种层次化概念引入了单个进程中，每个进程都指定一个唯一的命名空间。因为每个进程都会继承父进程的命名空间（除非是特别声明的情况），所以所有进程往往都只有一个全局命名空间。

么要想使用 VFS，就必须将目录重新表示为文件形式。通常，这种转换需要在使用现场（on the fly）引入一些特殊处理，使得非 Unix 文件系统能够兼容 Unix 文件系统的使用规则并满足 VFS 的需求。这种文件系统当然仍能工作，但是其带来的开销则不可思议（开销太大了）。

13.4 VFS 对象及其数据结构

VFS 其实采用的是面向对象^①的设计思路，使用一组数据结构来代表通用文件对象。这些数据结构类似于对象。因为内核纯粹使用 C 代码实现，没有直接利用面向对象的语言，所以内核中的数据结构都使用 C 语言的结构体实现，而这些结构体包含数据的同时也包含操作这些数据的函数指针，其中的操作函数由具体文件系统实现。

VFS 中有四个主要的对象类型，它们分别是：

- 超级块对象，它代表一个具体的已安装文件系统。
- 索引节点对象，它代表一个具体文件。
- 目录项对象，它代表一个目录项，是路径的一个组成部分。
- 文件对象，它代表由进程打开的文件。

注意，因为 VFS 将目录作为一个文件来处理，所以不存在目录对象。回忆本章前面所提到的目录项代表的是路径中的一个组成部分，它可能包括一个普通文件。换句话说，目录项不同于目录，但目录却是另一种形式的文件，明白了吗？

每个主要对象中都包含一个操作对象，这些操作对象描述了内核针对主要对象可以使用的方法：

- `super_operations` 对象，其中包括内核针对特定文件系统所能调用的方法，比如 `write_inode()` 和 `sync_fs()` 等方法。
- `inode_operations` 对象，其中包括内核针对特定文件所能调用的方法，比如 `create()` 和 `link()` 等方法。
- `dentry_operations` 对象，其中包括内核针对特定目录所能调用的方法，比如 `d_compare()` 和 `d_delete()` 等方法。
- `file_operations` 对象，其中包括进程针对已打开文件所能调用的方法，比如 `read()` 和 `write()` 等方法。

操作对象作为一个结构体指针来实现，此结构体中包含指向操作其父对象的函数指针。对于其中许多方法来说，可以继承使用 VFS 提供的通用函数，如果通用函数提供的基本功能无法满足需要，那么就必须使用实际文件系统的独有方法填充这些函数指针，使其指向文件系统实例。

再次提醒，我们这里所说的对象就是指结构体，而不是像 C++ 或 Java 那样的真正的对象数据类型。但是这些结构体的确代表的是一个对象，它含有相关的数据和对这些数据的操作，所以可以说它们就是对象。

① 人们时常忽略，甚至会否认，但是在内核中确实存在很多利用面向对象思想编程的例子。虽然内核开发者可能有意避免 C++ 和其他面向对象语言，但是面向对象的思想仍然经常被借鉴——虽然 C 语言缺乏面向对象的机制。VFS 就是一个利用 C 代码来有效和简洁地实现 OOP 的例子。

VFS 使用了大量结构体对象，它所包括的对象远远多于上面提到的这几种主要对象。比如每个注册的文件系统都由 `file_system_type` 结构体来表示，它描述了文件系统及其性能；另外，每一个安装点也都用 `vfsmount` 结构体表示，它包含的是安装点的相关信息，如位置和安装标志等。

在本章的最后还要介绍两个与进程相关的结构体，它们描述了文件系统以及和进程相关的文件，分别是 `fs_struct` 结构体和 `file` 结构体。

13.5 节将讨论这些对象以及它们在 VFS 层的实现中扮演的角色。

13.5 超级块对象

各种文件系统都必须实现超级块对象，该对象用于存储特定文件系统的信息，通常对应于存放在磁盘特定扇区中的文件系统超级块或文件系统控制块（所以称为超级块对象）。对于并非基于磁盘的文件系统（如基于内存的文件系统，比如 `sysfs`），它们会在使用现场创建超级块并将其保存到内存中。

超级块对象由 `super_block` 结构体表示，定义在文件 `<linux/fs.h>` 中，下面给出它的结构和各个域的描述：

```
struct super_block {
    struct list_head    s_list;           /* 指向所有超级块的链表 */
    dev_t               s_dev;           /* 设备标识符 */
    unsigned long       s_blocksize;     /* 以字节为单位的块大小 */

    unsigned char       s_blocksize_bits; /* 以位为单位的块大小 */
    unsigned char       s_dirt;          /* 修改（脏）标志 */
    unsigned long long  s_maxbytes;      /* 文件大小上限 */
    struct file_system_type s_type;       /* 文件系统类型 */
    struct super_operations s_op;         /* 超级块方法 */
    struct dquot_operations *dq_op;      /* 磁盘限额方法 */
    struct quotactl_ops  *s_qcop;        /* 限额控制方法 */
    struct export_operations *s_export_op; /* 导出方法 */
    unsigned long       s_flags;         /* 挂载标志 */
    unsigned long       s_magic;         /* 文件系统的幻数 */
    struct dentry        *s_root;        /* 目录挂载点 */
    struct rw_semaphore  s_umount;       /* 卸载信号量 */
    struct semaphore     s_lock;         /* 超级块信号量 */
    int                  s_count;        /* 超级块引用计数 */
    int                  s_need_sync;    /* 尚未同步标志 */
    atomic_t             s_active;       /* 活动引用计数 */
    void                 *s_security;    /* 安全模块 */
    struct xattr_handler **s_xattr;     /* 扩展的属性操作 */
    struct list_head     s_inodes;       /* inodes 链表 */
    struct list_head     s_dirty;        /* 脏数据链表 */
    struct list_head     s_io;           /* 回写链表 */
    struct list_head     s_more_io;      /* 更多回写的链表 */
    struct hlist_head     s_anon;        /* 匿名目录项 */
    struct list_head     s_files;        /* 被分配文件链表 */
    struct list_head     s_dentry_lru;   /* 未被使用目录项链表 */
    int                  s_nr_dentry_unused; /* 链表中目录项的数目 */
    struct block_device  *s_bdev;        /* 相关的块设备 */
}
```

```

struct mtd_info          *s_mtd;          /* 存储磁盘信息 */
struct list_head         s_instances;     /* 该类型文件系统 */
struct quota_info        s_dquot;        /* 限额相关选项 */
int                      s_frozen;        /* frozen 标志位 */
wait_queue_head_t        s_wait_unfrozen; /* 冻结的等待队列 */
char                     s_id[32];        /* 文本名字 */
void                     *s_fs_info;       /* 文件系统特殊信息 */
fmode_t                  s_mode;          /* 安装权限 */
struct semaphore          s_vfs_rename_sem; /* 重命名信号量 */
u32                      s_time_gran;     /* 时间戳粒度 */
char                     *s_subtype;      /* 子类型名称 */
char                     *s_options;      /* 已存安装选项 */
};

```

创建、管理和撤销超级块对象的代码位于文件 `fs/super.c` 中。超级块对象通过 `alloc_super()` 函数创建并初始化。在文件系统安装时，文件系统会调用该函数以便从磁盘读取文件系统超级块，并且将其信息填充到内存中的超级块对象中。

13.6 超级块操作

超级块对象中最重要的一個域是 `s_op`，它指向超级块的操作函数表。超级块操作函数表由 `super_operations` 结构体表示，定义在文件 `<linux/fs.h>` 中，其形式如下：

```

struct super_operations {
    struct inode *(*alloc_inode)(struct super_block *sb);
    void (*destroy_inode)(struct inode *);
    void (*dirty_inode) (struct inode *);
    int (*write_inode) (struct inode *, int);
    void (*drop_inode) (struct inode *);
    void (*delete_inode) (struct inode *);
    void (*put_super) (struct super_block *);
    void (*write_super) (struct super_block *);
    int (*sync_fs)(struct super_block *sb, int wait);
    int (*freeze_fs) (struct super_block *);
    int (*unfreeze_fs) (struct super_block *);
    int (*statfs) (struct dentry *, struct kstatfs *);
    int (*remount_fs) (struct super_block *, int *, char *);
    void (*clear_inode) (struct inode *);
    void (*umount_begin) (struct super_block *);
    int (*show_options)(struct seq_file *, struct vfsmount *);
    int (*show_stats)(struct seq_file *, struct vfsmount *);
    ssize_t (*quota_read)(struct super_block *, int, char *, size_t, loff_t);
    ssize_t (*quota_write)(struct super_block *, int, const char *, size_t, loff_t);
    int (*bdev_try_to_free_page)(struct super_block *, struct page *, gfp_t);
};

```

该结构体中的每一项都是一个指向超级块操作函数的指针，超级块操作函数执行文件系统和索引节点的低层操作。

当文件系统需要对其超级块执行操作时，首先要在超级块对象中寻找需要的操作方法。比

如，如果一个文件系统要写自己的超级块，需要调用：

```
sb->s_op->write_super(sb);
```

在这个调用中，sb 是指向文件系统超级块的指针，沿着该指针进入超级块操作函数表 s_op，并从表中取得希望得到的 write_super() 函数，该函数执行写入超级块的实际操作。注意，尽管 write_super() 方法来自超级块，但是在调用时，还是要把超级块作为参数传递给它，这是因为 C 语言中缺少对面向对象的支持，而在 C++ 中，使用如下的调用就足够了：

```
sb.write_super();
```

由于在 C 语言中无法直接得到操作函数的父对象，所以必须将父对象以参数形式传给操作函数。下面给出 super_operation 中，超级块操作函数的用法。

- struct inode *alloc_inode(struct super_block *sb)

在给定的超级块下创建和初始化一个新的索引节点对象。

- void destroy_inode(struct inode *inode)

用于释放给定的索引节点。

- void dirty_inode(struct inode *inode)

VFS 在索引节点脏（被修改）时会调用此函数。日志文件系统（如 ext3 和 ext4）执行该函数进行日志更新。

- void write_inode(struct inode *inode, int wait)

用于将给定的索引节点写入磁盘。wait 参数指明写操作是否需要同步。

- void drop_inode(struct inode *inode)

在最后一个指向索引节点的引用被释放后，VFS 会调用该函数。VFS 只需要简单地删除这个索引节点后，普通 Unix 文件系统就不会定义这个函数了。

- void delete_inode(struct inode *inode)

用于从磁盘上删除给定的索引节点。

- void put_super(struct super_block *sb)

在卸载文件系统时由 VFS 调用，用来释放超级块。调用者必须一直持有 s_lock 锁。

- void write_super(struct super_block *sb)

用给定的超级块更新磁盘上的超级块。VFS 通过该函数对内存中的超级块和磁盘中的超级块进行同步。调用者必须一直持有 s_lock 锁。

- int sync_fs(struct super_block *sb, int wait)

使文件系统的数据元与磁盘上的文件系统同步。wait 参数指定操作是否同步。

- void write_super_lockfs(struct super_block *sb)

首先禁止对文件系统做改变，再使用给定的超级块更新磁盘上的超级块。目前 LVM(逻辑卷标管理)会调用该函数。

- void unlockfs(struct super_block *sb)

对文件系统解除锁定，它是 write_super_lockfs() 的逆操作。

- int statfs(struct super_block *sb, struct statfs *statfs)

VFS 通过调用该函数获取文件系统状态。指定文件系统相关的统计信息将放置在 statfs 中。

- int remount_fs(struct super_block *sb, int *flags, char *data)

当指定新的安装选项重新安装文件系统时，VFS 会调用该函数。调用者必须一直持有 s_lock 锁。

- void clear_inode(struct inode *inode)

VFS 调用该函数释放索引节点，并清空包含相关数据的所有页面。

- void umount_begin(struct super_block *sb)

VFS 调用该函数中断安装操作。该函数被网络文件系统使用，如 NFS。

所有以上函数都是由 VFS 在进程上下文中调用。除了 dirty_inode(), 其他函数在必要时都可以阻塞。

这其中的一些函数是可选的。在超级块操作表中，文件系统可以将不需要的函数指针设置成 NULL。如果 VFS 发现操作函数指针是 NULL，那它要么就会调用通用函数执行相应操作，要么什么也不做，如何选择取决于具体操作。

13.7 索引节点对象

索引节点对象包含了内核在操作文件或目录时需要的全部信息。对于 Unix 风格的文件系统来说，这些信息可以从磁盘索引节点直接读入。如果一个文件系统没有索引节点，那么，不管这些相关信息在磁盘上是怎么存放的，文件系统都必须从中提取这些信息。没有索引节点的文件系统通常将文件的描述信息作为文件的一部分来存放。这些文件系统与 Unix 风格的文件系统不同，没有将数据与控制信息分开存放。有些现代文件系统使用数据库来存储文件的数据。不管哪种情况、采用哪种方式，索引节点对象必须在内存中创建，以便于文件系统使用。

索引节点对象由 inode 结构体表示，它定义在文件 <linux/fs.h> 中，下面给出它的结构体和各项的描述。

```
struct inode {
    struct hlist_node    i_hash;           /* 散列表 */
    struct list_head     i_list;           /* 索引节点链表 */
    struct list_head     i_sb_list;       /* 超级块链表 */
    struct list_head     i_dentry;        /* 目录项链表 */
    unsigned long         i_ino;          /* 节点号 */
}
```

```

atomic_t          i_count;          /* 引用计数 */
unsigned int       i_nlink;          /* 硬链接数 */
uid_t             i_uid;             /* 使用者的 id */
gid_t             i_gid;             /* 使用组的 id */
kdev_t            i_rdev;            /* 实际设备标识符 */
u64               i_version;         /* 版本号 */
loff_t            i_size;            /* 以字节为单位的文件大小 */
seqcount_t        i_size_seqcount;  /* 对 i_size 进行串行计数 */
struct timespec   i_atime;           /* 最后访问时间 */
struct timespec   i_mtime;           /* 最后修改时间 */
struct timespec   i_ctime;           /* 最后改变时间 */
unsigned int       i_blkbits;         /* 以位为单位的块大小 */
blkcnt_t          i_blocks;          /* 文件的块数 */
unsigned short     i_bytes;           /* 使用的字节数 */
umode_t           i_mode;            /* 访问权限 */
spinlock_t        i_lock;            /* 自旋锁 */
struct rw_semaphore i_alloc_sem;     /* 嵌入 i_sem 内部 */
struct semaphore   i_sem;            /* 索引节点信号量 */
struct inode_operations *i_op;       /* 索引节点操作表 */
struct file_operations *i_fop;       /* 缺省的索引节点操作 */
struct super_block *i_sb;            /* 相关的超级块 */
struct file_lock   *i_flock;         /* 文件锁链表 */
struct address_space *i_mapping;     /* 相关的地址映射 */
struct address_space i_data;         /* 设备地址映射 */
struct dquot       *i_dquot[MAXQUOTAS]; /* 索引节点的磁盘限额 */
struct list_head    i_devices;       /* 块设备链表 */
union {
    struct pipe_inode_info *i_pipe;   /* 管道信息 */
    struct block_device    *i_bdev;   /* 块设备驱动 */
    struct cdev            *i_cdev;   /* 字符设备驱动 */
};
unsigned long        i_dnotify_mask; /* 目录通知掩码 */
struct dnotify_struct *i_dnotify;     /* 目录通知 */
struct list_head     inotify_watches; /* 索引节点通知监测链表 */
struct mutex         inotify_mutex;    /* 保护 inotify_watches */
unsigned long        i_state;          /* 状态标志 */
unsigned long        dirtied_when;     /* 第一次弄脏数据的时间 */
unsigned int         i_flags;          /* 文件系统标志 */
atomic_t             i_writecount;     /* 写者计数 */
void                *i_security;       /* 安全模块 */
void                *i_private;        /* fs 私有指针 */
};

```

一个索引节点代表文件系统中（但是索引节点仅当文件被访问时，才在内存中创建）的一个文件，它也可以是设备或管道这样的特殊文件。因此索引节点结构体中有一些和特殊文件相关的项，比如 `i_pipe` 项就指向一个代表有名管道的数据结构，`i_bdev` 指向块设备结构体，`i_cdev` 指向字符设备结构体。这三个指针被存放在一个公用体中，因为一个给定的索引节点每次只能表示三者之一（或三者均不）。

有时，某些文件系统可能并不能完整地包含索引节点结构体要求的所有信息。举个例子，有的文件系统可能并不记录文件的访问时间，这时，该文件系统就可以在实现中选择任意合适的办法来解决这个问题。它可以在 `i_atime` 中存储 0，或者让 `i_atime` 等于 `i_mtime`，或者只在内存中更新 `i_atime` 而不将其写回磁盘，或者由文件系统的实现者来决定。

13.8 索引节点操作

和超级块操作一样，索引节点对象中的 `inode_operations` 项也非常重要，因为它描述了 VFS 用以操作索引节点对象的所有方法，这些方法由文件系统实现。与超级块类似，对索引节点的操作调用方式如下：

```
i->i_op->truncate(i)
```

`i` 指向给定的索引节点，`truncate()` 函数是由索引节点 `i` 所在的文件系统定义的。`inode_operations` 结构体定义在文件 `<linux/fs.h>` 中：

```
struct inode_operations {
    int (*create) (struct inode *, struct dentry *, int, struct nameidata *);
    struct dentry * (*lookup) (struct inode *, struct dentry *, struct nameidata *);
    int (*link) (struct dentry *, struct inode *, struct dentry *);
    int (*unlink) (struct inode *, struct dentry *);
    int (*symlink) (struct inode *, struct dentry *, const char *);
    int (*mkdir) (struct inode *, struct dentry *, int);
    int (*rmdir) (struct inode *, struct dentry *);
    int (*mknod) (struct inode *, struct dentry *, int, dev_t);
    int (*rename) (struct inode *, struct dentry *,
                  struct inode *, struct dentry *);
    int (*readlink) (struct dentry *, char __user *, int);
    void * (*follow_link) (struct dentry *, struct nameidata *);
    void (*put_link) (struct dentry *, struct nameidata *, void *);
    void (*truncate) (struct inode *);
    int (*permission) (struct inode *, int);
    int (*setattr) (struct dentry *, struct iattr *);
    int (*getattr) (struct vfsmount *mnt, struct dentry *, struct kstat *);
    int (*setxattr) (struct dentry *, const char *, const void *, size_t, int);
    ssize_t (*getxattr) (struct dentry *, const char *, void *, size_t);
    ssize_t (*listxattr) (struct dentry *, char *, size_t);
    int (*removexattr) (struct dentry *, const char *);
    void (*truncate_range) (struct inode *, loff_t, loff_t);
    long (*fallocate) (struct inode *inode, int mode, loff_t offset,
                      loff_t len);
    int (*fiemap) (struct inode *, struct fiemap_extent_info *, u64 start,
                  u64 len);
};
```

下面这些接口由各种函数组成，在给定的节点上，可能由 VFS 执行这些函数，也可能由具体的文件系统执行：

- `int create(struct inode *dir, struct dentry *dentry, int mode)`

VFS 通过系统调用 `create()` 和 `open()` 来调用该函数，从而为 `dentry` 对象创建一个新的索引节点。在创建时使用 `mode` 指定的初始模式。

- `struct dentry * lookup(struct inode *dir, struct dentry *dentry)`

该函数在特定目录中寻找索引节点，该索引节点要对应于 dentry 中给出的文件名。

- `int link(struct dentry *old_dentry,
 struct inode *dir,
 struct dentry *dentry)`

该函数被系统调用 `link()` 调用，用来创建硬连接。硬连接名称由 `dentry` 参数指定，连接对象是 `dir` 目录中 `old_dentry` 目录项所代表的文件。

- `int unlink(struct inode *dir, struct dentry *dentry)`

该函数被系统调用 `unlink()` 调用，从目录 `dir` 中删除由目录项 `dentry` 指定的索引节点对象。

- `int symlink(struct inode *dir,
 struct dentry *dentry,
 const char *symname)`

该函数被系统调用 `symlink()` 调用，创建符号连接。该符号连接名称由 `symname` 指定，连接对象是 `dir` 目录中的 `dentry` 目录项。

- `int mkdir(struct inode *dir,
 struct dentry *dentry,
 int mode)`

该函数被系统调用 `mkdir()` 调用，创建一个新目录。创建时使用 `mode` 指定的初始模式。

- `int rmdir(struct inode *dir,
 struct dentry *dentry)`

该函数被系统调用 `rmdir()` 调用，删除 `dir` 目录中的 `dentry` 目录项代表的文件。

- `int mknod(struct inode *dir,
 struct dentry *dentry,
 int mode, dev_t rdev)`

该函数被系统调用 `mknod()` 调用，创建特殊文件（设备文件、命名管道或套接字）。要创建的文件放在 `dir` 目录中，其目录项为 `dentry`，关联的设备为 `rdev`，初始权限由 `mode` 指定。

- `int rename(struct inode *old_dir,
 struct dentry *old_dentry,
 struct inode *new_dir,
 struct dentry *new_dentry)`

VFS 调用该函数来移动文件。文件源路径在 `old_dir` 目录中，源文件由 `old_dentry` 目录项指定，目标路径在 `new_dir` 目录中，目标文件由 `new_dentry` 指定。

- `int readlink(struct dentry *dentry,
 char *buffer, int buflen)`

该函数被系统调用 `readlink()` 调用，拷贝数据到特定的缓冲 `buffer` 中。拷贝的数据来自 `dentry` 指定的符号连接，拷贝大小最大可达 `buflen` 字节。

- `int follow_link(struct dentry *dentry,
 struct nameidata *nd)`

该函数由 VFS 调用，从一个符号连接查找它指向的索引节点。由 dentry 指向的连接被解析，其结果存放在由 nd 指向的 nameidata 结构体中。

- int put_link(struct dentry *dentry,
 struct nameidata *nd)

在 follow_link() 调用之后，该函数由 VFS 调用进行清除工作。

- void truncate(struct inode *inode)

该函数由 VFS 调用，修改文件的大小。在调用前，索引节点的 i_size 项必须设置为预期的大小。

- int permission(struct inode *inode, int mask)

该函数用来检查给定的 inode 所代表的文件是否允许特定的访问模式。如果允许特定的访问模式，返回零，否则返回负值的错误码。多数文件系统都将此区域设置为 NULL，使用 VFS 提供的通用方法进行检查。这种检查操作仅仅比较索引节点对象中的访问模式位是否和给定的 mask 一致。比较复杂的系统（比如支持访问控制链（ACLS）的文件系统），需要使用特殊的 permission() 方法。

- int setattr(struct dentry *dentry,
 struct iattr *attr)

该函数被 notify_change() 调用，在修改索引节点后，通知发生了“改变事件”。

- int getattr(struct vfsmount *mnt,
 struct dentry *dentry,
 struct kstat *stat)

在通知索引节点需要从磁盘中更新时，VFS 会调用该函数。

扩展属性允许 key/value 这样的一对值与文件相关联。

- int setxattr(struct dentry *dentry,
 const char *name,
 const void *value,
 size_t size, int flags)

该函数由 VFS 调用，给 dentry 指定的文件设置扩展属性。属性名为 name，值为 value。

- ssize_t getxattr(struct dentry *dentry,
 const char *name,
 void *value, size_t size)

该函数由 VFS 调用，向 value 中拷贝给定文件的扩展属性 name 对应的数值。

- ssize_t listxattr(struct dentry *dentry,
 char *list, size_t size)

该函数将特定文件的所有属性列表拷贝到一个缓冲列表中。

- int removexattr(struct dentry *dentry,
 const char *name)

该函数从给定文件中删除指定的属性。

13.9 目录项对象

VFS 把目录当作文件对待，所以在路径 `/bin/vi` 中，`bin` 和 `vi` 都属于文件——`bin` 是特殊的目录文件而 `vi` 是一个普通文件，路径中的每个组成部分都由一个索引节点对象表示。虽然它们可以统一由索引节点表示，但是 VFS 经常需要执行目录相关的操作，比如路径名查找等。路径名查找需要解析路径中的每一个组成部分，不但要确保它有效，而且还需要再进一步寻找路径中的下一个部分。

为了方便查找操作，VFS 引入了目录项的概念。每个 `dentry` 代表路径中的一个特定部分。对前一个例子来说，`/`、`bin` 和 `vi` 都属于目录项对象。前两个是目录，最后一个是普通文件。必须明确一点：在路径中（包括普通文件在内），每一个部分都是目录项对象。解析一个路径并遍历其分量绝非简单的演练，它是耗时的、常规的字符串比较过程，执行耗时、代码繁琐。目录项对象的引入使得这个过程更加简单。

目录项也可包括安装点。在路径 `/mnt/cdrom/foo` 中，构成元素 `/`、`mnt`、`cdrom` 和 `foo` 都属于目录项对象。VFS 在执行目录操作时（如果需要的话）会现场创建目录项对象。

目录项对象由 `dentry` 结构体表示，定义在文件 `<linux/dcache.h>` 中。下面给出该结构体和其中各项的描述：

```
struct dentry {
    atomic_t          d_count;           /* 使用记数 */
    unsigned int      d_flags;           /* 目录项标识 */
    spinlock_t        d_lock;           /* 单目录项锁 */
    int               d_mounted;         /* 是登录点的目录项吗？ */
    struct inode       *d_inode;         /* 相关联的索引节点 */
    struct hlist_node d_hash;           /* 散列表 */
    struct dentry      *d_parent;        /* 父目录的目录项对象 */
    struct qstr        d_name;           /* 目录项名称 */
    struct list_head   d_lru;           /* 未使用的链表 */
    union {
        struct list_head d_child;       /* 目录项内部形成的链表 */
        struct rcu_head  d_rcu;         /* RCU 加锁 */
    } d_u;
    struct list_head   d_subdirs;        /* 子目录链表 */
    struct list_head   d_alias;          /* 索引节点别名链表 */
    unsigned long      d_time;           /* 重置时间 */
    struct dentry_operations *d_op;      /* 目录项操作指针 */
    struct super_block *d_sb;            /* 文件的超级块 */
    void               *d_fsdata;        /* 文件系统特有数据 */
    unsigned char      d_iname[DNAME_INLINE_LEN_MIN]; /* 短文件名 */
};
```

与前面的两个对象不同，目录项对象没有对应的磁盘数据结构，VFS 根据字符串形式的路径名现场创建它。而且由于目录项对象并非真正保存在磁盘上，所以目录项结构体没有是否被修改的标志（也就是是否为脏、是否需要写回磁盘的标志）。

13.9.1 目录项状态

目录项对象有三种有效状态：被使用、未被使用和负状态。

一个被使用的目录项对应一个有效的索引节点（即 `d_inode` 指向相应的索引节点）并且表明该对象存在一个或多个使用者（即 `d_count` 为正值）。一个目录项处于被使用状态，意味着它正被 VFS 使用并且指向有效的数据，因此不能被丢弃。

一个未被使用的目录项对应一个有效的索引节点（`d_inode` 指向一个索引节点），但是应指明 VFS 当前并未使用它（`d_count` 为 0）。该目录项对象仍然指向一个有效对象，而且被保留在缓存中以便需要时再使用它。由于该目录项不会过早地被撤销，所以以后再需要它时，不必重新创建，与未缓存的目录项相比，这样使路径查找更迅速。但如果要回收内存的话，可以撤销未使用的目录项。

一个负状态的目录项^①没有对应的有效索引节点（`d_inode` 为 NULL），因为索引节点已被删除了，或路径不再正确了，但是目录项仍然保留，以便快速解析以后的路径查询。比如，一个守护进程不断地去试图打开并读取一个不存在的配置文件。`open()` 系统调用不断地返回 `ENOENT`，直到内核构建了这个路径、遍历磁盘上的目录结构体并检查这个文件的确不存在为止。即便这个失败的查找很浪费资源，但是将负状态缓存起来还是非常值得的。虽然负状态的目录项有些用处，但是如果有需要，可以撤销它，因为毕竟实际上很少用到它。

目录项对象释放后也可以保存到 slab 对象缓存中去，这点在第 12 章讨论过。此时，任何 VFS 或文件系统代码都没有指向该目录项对象的有效引用。

13.9.2 目录项缓存

如果 VFS 层遍历路径名中所有的元素并将它们逐个地解析成目录项对象，还要到达最深层目录，将是一件非常费力的工作，会浪费大量的时间。所以内核将目录项对象缓存在目录项缓存（简称 `dcache`）中。

目录项缓存包括三个主要部分：

- “被使用的”目录项链表。该链表通过索引节点对象中的 `i_dentry` 项连接相关的索引节点，因为一个给定的索引节点可能有多个链接，所以就可能有多个目录项对象，因此用一个链表来连接它们。
- “最近被使用的”双向链表。该链表含有未被使用的和负状态的目录项对象。由于该链总是在头部插入目录项，所以链头节点的数据总比链尾的数据要新。当内核必须通过删除节点项回收内存时，会从链尾删除节点项，因为尾部的节点最旧，所以它们在近期内再次被使用的可能性最小。
- 散列表和相应的散列函数用来快速地将给定路径解析为相关目录项对象。

散列表由数组 `dentry_hashtable` 表示，其中每一个元素都是一个指向具有相同键值的目录项对象链表的指针。数组的大小取决于系统中物理内存的大小。

实际的散列值由 `d_hash()` 函数计算，它是内核提供给文件系统的唯一的一个散列函数。

查找散列表要通过 `d_lookup()` 函数，如果该函数在 `dcache` 中发现了与其相匹配的目录项对象，则匹配的对象被返回；否则，返回 NULL 指针。

^① 这个名字容易产生误导，其实它和任何负数或负状态并没有联系。更准确的名称应该是无效目录项。